Content-Moderated Bilateral Access Control for Privacy-Preserving Cloud Data Sharing Services

Chao Wang, Willy Susilo, *Fellow, IEEE*, Yudi Zhang, Yumei Li, and Qinlong Huang, *Member, IEEE*

***Abstract*—Cloud computing facilitates scalable data sharing across multiple organizations and users, but also raises concerns about data privacy. Matchmaking encryption (ME) is a prominent technique that enforces bilateral access control in cloud services such as cloud marketplace, allowing both senders and receivers to specify policies for the encrypted data to be revealed. However, receivers may be at risk of being exposed to malicious or harmful content, thus undermining their trust in cloud service platforms. To this end, we introduce MBAC, a content-moderated bilateral access control framework for privacy-preserving cloud data sharing services, which allows receivers to acquire data from authentic senders while preserving their anonymity, and report malicious content in a verifiable manner, i.e., empowering the service provider to hold senders accountable. MBAC is built upon a novel primitive called franking broadcast ME (FBME), which generates a franking signature for the data by designating the service provider as the moderator to ensure accountability and deniability, and encrypts both the data and its franking signature while embedding the sender secret key for privacy and authenticity. We then present a concrete construction of FBME from key-private public key encryption, strongly unforgeable one-time signature and non-interactive zero-knowledge proof. Formal security analysis and extensive experiments demonstrate that MBAC provides efficient bilateral access control and content moderation for cloud data sharing services.**

***Index Terms*—Matchmaking encryption, anonymity, content moderation, message franking, cloud computing.**

I. Introduction

CLOUD computing has transformed the way organizations and users store and share data, providing scalable services with minimal maintenance costs. In particular, cloud-based data marketplaces such as Snowflake and Datarade, hosted on cloud infrastructures, have emerged as essential platforms for data exchange between providers and consumers. However, the openness of such infrastructures introduces serious security and privacy concerns. In light of data protection and quality regulations like the GDPR, secure and privacy-preserving data sharing has become indispensable. To address these needs, we identify three fundamental security requirements for cloud-based data sharing:

- *Privacy Protection:* To protect sensitive information and meet regulatory requirements, data must be encrypted before being stored on semi-trusted cloud platforms. Beyond data confidentiality, privacy further demands sender and receiver anonymity: unauthorized entities should not learn the sender's identity, and even authorized receivers should not infer who else is in the recipient set.
- *Bilateral Access Control:* As shown in Fig. 1, both parties typically wish to impose access restrictions: providers intend to share data only with authorized receivers, while consumers prefer to accept data only from certified sources. This mutual constraint gives rise to the bilateral access control paradigm, where access is granted only when both sender and receiver policies are satisfied.
- *Content Moderation:* Cloud data sharing platforms are increasingly exploited for distributing untrustworthy or deceptive information, such as biased analytics, spam advertising, and fraudulent content. Effective encrypted content moderation is therefore crucial, allowing authorized receivers to report suspected abuse to the service provider (moderator) for verification and appropriate action.

Traditional cryptographic schemes such as public key encryption (PKE) [1], broadcast encryption (BE) [2], and PKE with keyword search (PEKS) [3] focus solely on receiver-side access control, lacking support for sender authentication or bilateral policy enforcement. To bridge this gap, bilateral access control mechanisms have emerged as a critical enhancement to ensure both information privacy and source identification. For instance, public-key authenticated encryption with keyword search (PAEKS) [4] allows senders to encrypt keywords using their secret keys and receivers' public keys, ensuring only authenticated keywords can be retrieved. However, PAEKS

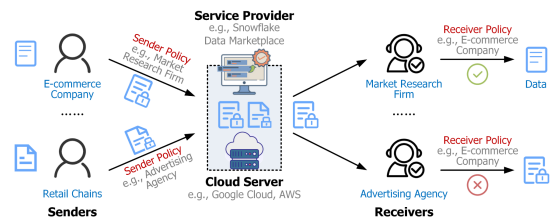


Fig. 1. Cloud data sharing services.

Received 10 March 2025; revised 14 July 2025; accepted 15 September 2025. Date of publication 23 September 2025; date of current version 8 December 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62572073, Grant 62472047 and Grant U1736212, in part by the Research Program of Beijing University of Posts and Telecommunications under Grant 2025TSQY06, in part by China Scholarship Council under Grant 202306470067, and in part by Australian Research Council Laureate Fellowship under Grant FL230100033. Recommended for acceptance by C. A. Ardagna. *(Corresponding author: Qinlong Huang.)*

Chao Wang and Qinlong Huang are with the School of Cyberspace Security, Beijing University of Posts and Telecommunications, Beijing 100876, China, and also with the National Engineering Research Center of Disaster Backup and Recovery, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: wang_chao@bupt.edu.cn; longsec@bupt.edu.cn).

Willy Susilo, Yudi Zhang, and Yumei Li are with the Institute of Cybersecurity and Cryptology, School of Computing and Information Technology, University of Wollongong, Wollongong, NSW 2522, Australia (e-mail: wsusilo@uow.edu.au; yudi@uow.edu.au; yumeil@uow.edu.au).

Digital Object Identifier 10.1109/TCC.2025.3612322

2168-7161 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:12:53 UTC from IEEE Xplore. Restrictions apply.

(a) SMF (b) AMF

Fig. 2. Message franking for encrypted content moderation.

is limited to keyword privacy and lacks data confidentiality and receiver anonymity. Fortunately, matchmaking encryption (ME) [5] advances data security by allowing decryption only when both sender and receiver policies match to ensure source authenticity with strong privacy. In particular, the privacy notion in ME captures both data confidentiality and receiver anonymity, preventing unauthorized entities from inferring both the data content and privileged receivers. These intrinsic properties make ME well-suited for privacy-preserving cloud sharing, enabling intended receivers to selectively access desired data while remaining anonymous to outsiders. For instance, RBME [6] is a multi-receiver ME that ensures privacy and authenticity for data matching in mobile social networks with chosen-plaintext attacks (CPA) security, while PSME [7] improves robustness against chosen ciphertext attacks (CCA) for flexible one-to-many data sharing. On the content moderation front, message franking (MF) [8] provides cryptographic support for verifiable abuse reporting without compromising confidentiality, which is classified into symmetric MF (SMF) [9] and asymmetric MF (AMF) [10]. As shown in Fig. 2, SMF requires service providers to associate user identifiers with encrypted messages, whereas AMF allows users to hide their identifiers from the platform, offering enhanced metadata privacy. Building on AMF, Reclean [11] integrates it with PEKS [3] to create a searchable message-moderation system.

Despite these advances, a unified solution that simultaneously supports bilateral access control and encrypted content moderation in privacy-preserving cloud sharing remains a pressing yet underexplored issue, warranting further research.

A. Potential Solutions and Challenges

In this work, we aim to design a content-moderated bilateral access control for privacy-preserving cloud data sharing services. Towards content moderation, MF offers a natural and promising foundation. Among its variants, AMF offers two essential properties:

- *Accountability:* AMF offers a verifiable reporting mechanism to hold malicious senders accountable for the harmful content they distribute in the channel. This ensures that malicious senders cannot evade moderation and that receivers cannot submit fabricated reports to accuse innocent senders falsely.
- *Deniability:* In sensitive contexts such as financial data marketplaces, senders must be able to deny responsibility for distributed content if exposed publicly after a compromise. Deniability ensures that only the service provider can verify reports of harmful content, preventing external attribution. This feature shields senders from reputational damage by allowing them to plausibly deny authorship of leaked data in post-compromise scenarios.

A natural direction might involve combining the state-of-the-art CCA-secure multi-receiver ME [7] with AMF [10] to achieve both anonymous bilateral access control and efficient content moderation over the shared data in cloud services. However, extending AMF to multi-receiver scenarios introduces new concerns, particularly regarding the efficiency of content moderation for multiple receivers. Recently, asymmetric group MF (AGMF) [12] was proposed for one-to-many communication with smaller signature sizes compared to extending directly from AMF. This makes AGMF an alternative approach, as it can generate a franking signature from the underlying data, which is then encrypted using ME [7] for intended receivers. After decryption, authorized receivers can report harmful content to the moderator, who verifies the franking signature and takes appropriate action. However, such a straightforward combination poses technical challenges due to differences in underlying cryptographic structures.

- First, ME [7] operates in an identity-based setting, while AGMF [12] relies the public-key setting, complicating direct integration. Particularly, it is non-trivial to convert AGMF based on symmetric prime groups to ME [7] based on asymmetric prime order groups, as any change in public parameters may break security reduction.
- Second, AGMF [12] guarantees only weak receiver anonymity, as its franking signature includes encapsulated keys for all target receivers, enabling them to identify others within the same set. This compromises receiver anonymity of ME, which mandates that even legal receivers cannot infer the authorized set from the ciphertext.

Therefore, a secure bilateral access control solution with content moderation in the multi-receiver setting remains a challenge in cloud data sharing services, particularly in meeting desirable properties like privacy, sender authentication, accountability, and deniability, as outlined in Table I.

B. Contributions

In this paper, we address this challenge and propose MBAC, a content-moderated bilateral access control framework for privacy-preserving cloud data sharing services. MBAC enables authorized receivers to access data from verified senders, and allows them to report harmful content to the service provider for holding malicious senders accountable, while preserving the anonymity of privileged receivers from unauthorized entities. Specifically, we make the following contributions.

- We first introduce a franking broadcast ME (FBME) for MBAC, along with its formal definition and security models. FBME is a novel multi-receiver ME scheme with verifiable reporting. It generates authenticated ciphertexts for target receivers, ensuring that only ciphertexts from the intended sender can be decrypted, while preserving the privacy of other privileged receivers. FBME also allows receivers to report harmful data to the moderator, enforcing accountability on malicious senders while maintaining deniability in post-breach scenarios.
- We then present a concrete construction of FBME, which generates asymmetric franking signatures solely on the data without disclosing the privileged receiver set, utilizing the non-interactive zero-knowledge (NIZK) proof system. FBME encrypts both the data and franking signature for intended receivers using a key-private PKE and a strongly unforgeable (SUF) one-time signature to achieve receiver

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:12:53 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON WITH REPRESENTATIVE ENCRYPTION SCHEMES

Schemes	Privacy		Sender Authentication	Content Moderation		Multi-Receiver	Security
	Confidentiality	Anonymity		Accountability	Deniability		
PAEKS [4]	✘	✘	✔	✘	✘	✘	-
IBME [5]	✔	✔	✔	✘	✘	✘	IND-CPA
RBME [6]	✔	✔	✔	✘	✘	✔	IND-CPA
PSME [7]	✔	✔*	✔	✘	✘	✔	IND-CCA
Reclean [11]	✔	✘	✘	✔	✔	✘	IND-CPA
FBME	✔	✔	✔	✔	✔	✔	IND-CCA

Note: ✔* indicates receiver anonymity only (no sender anonymity). IND-CPA/IND-CCA denote ciphertext indistinguishability under CPA/CCA.

anonymity and CCA security, while embedding the sender secret key into the ciphertext under the computational bilinear Diffie-Hellman (CBDH) assumption for sender identification.

- We prove the security of FBME, covering privacy, authenticity, accountability and deniability under standard assumptions. We also implement FBME and relevant schemes to evaluate the feasibility of MBAC. Experimental results show that FBME achieves encryption efficiency comparable to existing schemes and verifiable reporting with minimal overhead, demonstrating MBAC's practicality with stronger security and reasonable performance.

C. Organization

The remainder of this paper is organized as follows. Section II reviews related work, followed by necessary preliminaries in Section III. We introduce the system model and formal definitions in Section IV, and describe the construction of MBAC based on FBME in Section V. The security analysis and performance evaluation are given in Section VI and VII. At last, we present a conclusion in Section VIII.

II. RELATED WORK

1) Bilateral Access Control: Ateniese et al. [5] introduced the notion of ME for privacy-aware policy matching, where an encrypted message can only be decrypted if both the sender's and receiver's policies are met, with no information leaked when decryption fails. They also presented the first identity-based ME (IBME) scheme based on the bilinear Diffie-Hellman (BDH) assumption in the random oracle model. Afterward, Chen et al. [13] put forward an IBME under the standard assumptions in the standard model, which is constructed from an anonymous identity-based encryption scheme and an anonymous signature scheme [14]. Zhang et. al [15] proposed IB-MFE by integrating functional encryption with IB-ME in dual pairing vector spaces, enabling the sender to authorize a receiver to compute specific functions over encrypted data. However, these schemes are limited to one-to-one settings, and then multi-receiver ME schemes have been developed. Wu et al. [6] proposed RBME in the random oracle model under the CBDH assumption, where the ciphertext does not reveal non-revoked receivers when a designated target receiver is removed. However, RBME's encryption cost increases quadratically with the number of receivers, and only provides CPA security. Sun et al. [7] introduced PSME, a CCA-secure multi-receiver ME under the DBDH assumption, which allows a sender to insert a signature into ciphertext and uses a broadcasting technique based on inner product encryption to conceal all target receivers.

In parallel, attribute-based ME variants have been explored for fine-grained bilateral access control. Wu et al. [16] presented a fuzzy IBME that derives encryption keys from the sender's attribute set and provides threshold-based matching between sender and receiver attributes under the decisional BDH (DBDH) and CBDH assumptions. Subsequently, Huang et al. [17] proposed an efficient fuzzy IB-ME with adaptive security in the generic group model for anonymous social networks, which employs a collision-resistant hash function for mapping attributes to support large universe and high scalability. Specially, Xu et al. [18] introduced matchmaking attribute-based encryption (MABE) supporting fine-grained bilateral access control and outsourced data source identification via edge nodes, with a concrete construction based on attribute-based encryption (ABE) and signature schemes in the random oracle model. Later, Xu et al. [19] proposed a server-aided revocable MABE for cloud-fog IoT systems, which additionally supports user revocation with publicly updatable ciphertexts and lightweight decryption suitable for resource-constrained devices. Ma et al. [20] proposed a novel MABE under honest-but-curious central authority, built upon ciphertext-policy ABE [21] and anonymous credential scheme (ACS), where ACS is responsible for generating the encryption key and achieving authenticity. While attribute-based approaches provide fine-grained policy expressiveness, they typically involve high pairing costs and complex policy evaluation. In contrast, our work follows the multi-receiver ME line, aiming for stronger receiver privacy and efficient content moderation in one-to-many communication scenarios.

2) Encrypted Content Moderation

Grubbs et al. [9] initiated the study of SMF by introducing compactly committing authenticated encryption with associated data (AEAD) as a cryptographic solution for verifiable abuse reporting. In AEAD, a portion of the ciphertext functions as a commitment to the message, revealed during decryption, while a verification algorithm checks the commitment's integrity. Later, Dodis et al. [22] identified flaws in Facebook's attachment franking scheme, which allowed malicious users to send unreportable messages, and proposed a secure and fast AEAD scheme using Merkle-Damgård-style hash functions to address non-committing encryption issues. However, these SMF schemes expose sender and/or receiver identities to the platform and lack deniability as defined in [8].

To address this, Tyagi et al. [10] proposed AMF for moderation in metadata-private settings, which formalizes deniability by ensuring that only the moderator can validate reports, while allowing the sender to deny responsibility for messages after a compromise. This is achieved through NIZK proof systems for a carefully crafted language based on the discrete logarithm

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:12:53 UTC from IEEE Xplore. Restrictions apply.

(DL) assumption and a non-standard assumption called the knowledge-of-exponent assumption. Issa et al. [23] enhanced AMF with a preprocessing model that provided forward and backward secrecy. Lai et al. [12] proposed AGMF for group messaging, where the sender generates a franking signature by signing the message and encapsulated keys of all intended receivers, enabling authorized receivers to verify and report the message. AGMF utilizes a specialized key encapsulation mechanism (KEM) under the decisional Diffie-Hellman (DDH) assumption, which can be interpreted as a hash proof system [24] supporting Sigma protocols to ensure efficient proof generation.

III. Preliminaries

A. Bilinear Pairing and Hardness Assumptions

Let $\mathbb{G}$, $\mathbb{H}$, and $\mathbb{G}_T$ be cyclic groups of prime order p. A map $e : \mathbb{G} \times \mathbb{H} \to \mathbb{G}_T$ is a bilinear pairing if it satisfies:

- *Bilinearity:* For all $g \in \mathbb{G}$, $h \in \mathbb{H}$, and $a, b \in \mathbb{Z}_p$, it holds that $e(g^a, h^b) = e(g, h)^{ab}$.
- *Non-degeneracy:* There exist g, h such that $e(g, h) \neq 1$.
- *Efficient Computability:* The computation of $e(g, h)$ can be performed in polynomial time.

If $\mathbb{G} = \mathbb{H}$, e is called a symmetric pairing; otherwise, it is asymmetric. An asymmetric pairing is of Type-III if there is no efficient isomorphism between $\mathbb{G}$ and $\mathbb{H}$ [25].

We now recall standard complexity assumptions:

Definition 1: (DL Assumption [12]) Let g be a random generator of $\mathbb{G}$. The DL assumption holds if for any probabilistic polynomial time (PPT) adversary $\mathcal{A}$, the advantage $\mathsf{Adv}^{\mathsf{DL}}_{\mathbb{G},\mathcal{A}}(\lambda) := \Pr[\mathcal{A}(g, g^x) = x]$ is negligible, where $x \in \mathbb{Z}_p^*$ is chosen at random.

Definition 2: (DDH Assumption [12]) The DDH assumption holds if for any PPT adversary $\mathcal{A}$, the advantage $\mathsf{Adv}^{\mathsf{DDH}}_{\mathbb{G},\mathcal{A}}(\lambda) := |\Pr[\mathcal{A}(g, g^x, g^y, Z = g^{xy}) = 1] - \frac{1}{2}|$ is negligible, where $x, y \in \mathbb{Z}_p^*$ are chosen randomly and Z is either g^{xy} or a random element in $\mathbb{G}$.

Definition 3: (CBDH Assumption [4]) Let $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$ be a bilinear map. The CBDH assumption holds if for any PPT adversary $\mathcal{A}$, the advantage $\mathsf{Adv}^{\mathsf{CBDH}}_{\mathbb{G},\mathcal{A}}(\lambda) = \Pr[\mathcal{A}(g, g^x, g^y, g^z) = e(g, g)^{xyz}]$ is negligible, where $x, y, z \in \mathbb{Z}_p^*$ are chosen randomly.

B. One-Time Signature

A one-time signature scheme [26] is a digital signature scheme designed to be securely used only once, i.e., to sign a single message, which consists of the following three algorithms:

- $\mathsf{KGen}(\lambda) \to (ak, vk)$: Generates a key pair consisting of a secret signing key ak and a public verification key vk.
- $\mathsf{Sign}(ak, m) \to \varsigma$: Produces a signature ς on message m using the secret key ak.
- $\mathsf{Vry}(vk, m, \varsigma) \to 0/1$: Verifies the validity of ς on m using the public key vk.

The scheme is strongly unforgeable if, after seeing a valid signature ς on message m, no PPT adversary can produce $(m', \varsigma') \neq (m, \varsigma)$ such that $\mathsf{Vry}(vk, m', \varsigma') = 1$. We also assume it is unlinkable (UL), i.e., vk reveals no information about the signer and is independent of any long-term key.

C. Kurosawa-Desmedt PKE Scheme

Kurosawa-Desmedt PKE (KDPKE) [27] is a KEM-based variant of the Cramer-Shoup encryption scheme [1], which is an efficient key-private PKE scheme with IND-CCA security in the standard model. The key privacy ensures that a ciphertext cannot be linked to any particular public key. Libert et al. [28] presented a randomness re-using KDPKE (RR-KDPKE) to reduce ciphertext size in multi-receiver PKE. It comprises the following four algorithms, where the randomness w can be re-used to compress ciphertexts in the multi-receiver setting.

- $\mathsf{Setup}(\lambda) \to pp$. Let $\mathbb{G}$ be a cyclic group of prime order p with random generators g_1, g_2. It chooses a collusion-resistant hash function $\mathsf{H}_1 : \{0,1\}^* \to \mathbb{Z}_p^*$ and a collusion-resistant key derivation function $\mathsf{H} : \mathbb{G} \to \{0,1\}^\eta$ for some $\eta \in \mathsf{poly}(\lambda)$. The public parameter is $pp = (\mathbb{G}, g_1, g_2, \mathsf{H}_1, \mathsf{H})$.
- $\mathsf{KGen}(pp) \to (pk, sk)$. It selects random exponents $x_1,\ x_2, y_1, y_2 \in \mathbb{Z}_p^*$ and computes $D_1 = g_1^{x_1} g_2^{x_2}, D_2 = g_1^{y_1} g_2^{y_2}$. The public key is $pk = (D_1, D_2)$, and the secret key is $sk = (x_1, x_2, y_1, y_2)$.
- $\mathsf{Enc}(pk, m) \to C$. It picks $w \in \mathbb{Z}_p^*$ and compute $V_1 = g_1^w, V_2 = g_2^w, K = (D_1 D_2^\alpha)^w$, where $\alpha = \mathsf{H}_1(V_1, V_2) \in \mathbb{Z}_p^*$. It then computes $A = m \oplus \mathsf{H}(K)$. The ciphertext is $C = (V_1, V_2, A)$.
- $\mathsf{Dec}(sk, C) \to m/\perp$. Parse C as (V_1, V_2, A). It computes $\alpha = H(V_1, V_2)$ and $K = V_1^{x_1+\alpha y_1} V_2^{x_2+\alpha y_2}$. Then, it returns $m = A \oplus \mathsf{H}(K)$ or $\perp$ if decryption fails.

RR-KDPKE is key-private and IND-CCA (AI-CCA) secure, weakly robust IND-CCA (WR-CCA) secure, and key-less reproducible [28]. Specifically, key-less reproducibility permits secure randomness re-use, which implies that there is a reproduction algorithm $\mathsf{Rep}(C, m', pk', sk')$ that takes as input a ciphertext $C = \mathsf{Enc}(pk, m; w)$, another message m', and a public/secret key pair (pk', sk'), and outputs a ciphertext that is equal to $\mathsf{Enc}(pk', m'; w)$ with probability 1.

D. NIZK Proof System

Let $\mathcal{L}$ represent a language and $\mathcal{R}$ be a relation such that every statement $\mathcal{V} \in \mathcal{L}$ has a corresponding witness with $(\mathcal{W}, \mathcal{V}) \in \mathcal{R}$. The NIZK proof system [12] for a witness-statement relation $\mathcal{R}$ is a pair of PPT algorithms.

- $\mathsf{Prove}(m, \mathcal{V}, \mathcal{W}) \to \pi$. Given a message m, a statement $\mathcal{V}$, and a witness $\mathcal{W}$ as input, this algorithm outputs a NIZK proof π.
- $\mathsf{Verify}(m, \mathcal{V}, \pi) \to 0/1$. Given m, $\mathcal{V}$, and a NIZK proof π as input, this algorithm outputs either 0 or 1.

The NIZK proof system features perfect soundness, knowledge extractability, and zero-knowledge [29]. Specifically, knowledge extractability ensures that if the statement is false, no cheating prover can convince the verifier it is true except with negligible probability, guaranteeing that the prover must know a valid witness. Zero-knowledge ensures that the proof reveals no information about the witness beyond its validity.

IV. MBAC Overview

A. System Model

As an access control framework for cloud data sharing, MBAC consists of five entities: the certificate authority (CA), the service

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:12:53 UTC from IEEE Xplore. Restrictions apply.

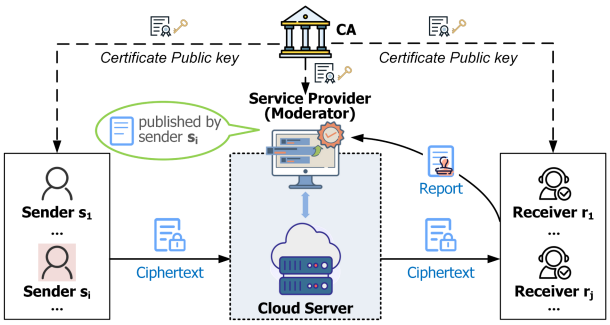


Fig. 3. System model of MBAC.

provider, the cloud server, data senders, and data receivers, as depicted in Fig. 3. The roles and functions of each entity are described as follows:

- *CA:* The CA is responsible for system initialization and public key certification. It generates global parameters and certifies users' public keys by issuing digital certificates after verifying their identities. These certificates allow others to verify the authenticity of public keys during encryption and decryption.
- *Service Provider:* The service provider operates as a data sharing platform (e.g., data marketplace) that facilitates interactions between data senders and receivers while outsourcing encrypted data storage to the cloud server. It also serves as a moderator, responsible for verifying reports of harmful content and holding the senders accountable.
- *Cloud Server:* The cloud server is a remote storage facility where the service provider stores encrypted data. It offers extensive storage capacity to accommodate large volumes of encrypted data uploaded by data senders through the service provider platform.
- *Data Senders:* Data senders obtain the certified public keys of the intended receivers from the CA, and then frank and encrypt the data using their own secret keys and the receivers' public keys before outsourcing the franked ciphertexts to the service provider.
- *Data Receivers:* Data receivers decrypt the franked ciphertext from the target sender. If they suspect the content is malicious, they can validate if it is reportable (i.e., whether the moderator can verify it) and then submit a report to the service provider for moderation.

In MBAC, the cloud server is considered semi-trusted, faithfully carrying out system operations but potentially curious about private information in the ciphertexts. All users and the moderator in the system generate their own public/secret key pairs, which are subsequently certified by the trusted CA to ensure the authenticity and integrity of their public keys.

B. Definition of FBME

MBAC is based on the notion of FBME, which allows data receivers to identify the authentic data sender and non-abusively report harmful content to ensure accountability while preserving sender deniability when necessary. Therefore, we first define the following algorithms of FBME for content-moderated bilateral access control. Let $\mathcal{U}$ be the set of all receivers in the system.

- $\mathsf{Setup}(\lambda, n) \rightarrow pp$. On input a security parameter λ and the maximal size n of the target set for one encryption, the algorithm outputs the public parameter pp.
- $\mathsf{SKGen}(pp) \rightarrow (spk, ssk)$. On input pp, it outputs a public/secret key pair (spk, ssk) for the data sender s.
- $\mathsf{RKGen}(pp) \rightarrow (rpk_j, rsk_j)$. On input pp, it outputs a public/secret key pair (rpk_j, rsk_j) for data receiver r_j.
- $\mathsf{JKGen}(pp) \rightarrow (pk_{\mathsf{J}}, sk_{\mathsf{J}})$. On input pp, it outputs a public/secret key pair $(pk_{\mathsf{J}}, sk_{\mathsf{J}})$ for the moderator.
- $\mathsf{Encrypt}(pp, pk_{\mathsf{J}}, spk, ssk, \mathcal{S}, m) \rightarrow ct$. On input pp, the moderator public key pk_{J}, a sender public key spk and secret key ssk, a target receiver set $\mathcal{S} \subseteq \mathcal{U}$, and a message m, the encryption algorithm outputs a ciphertext ct that encrypts m along with its franking signature σ.
- $\mathsf{Decrypt}(pp, pk_{\mathsf{J}}, spk, rsk_j, ct) \rightarrow (m, \sigma)/\perp$. On input pp, the moderator public key pk_{J}, a sender public key spk, a receiver secret key rsk_j, and a ciphertext ct, the decryption algorithm outputs either the plaintext and franking signature pair (m, σ) or $\perp$.
- $\mathsf{Report}(pp, pk_{\mathsf{J}}, spk, m, \sigma) \rightarrow (1, \mathsf{report})/0$. On input pp, the moderator public key pk_{J}, a sender public key spk, a message m and a franking signature σ, the algorithm outputs 0 if σ is invalid; otherwise, it outputs 1 and generates a report report that can be sent to the moderator for further judgment.
- $\mathsf{Judge}(pp, sk_{\mathsf{J}}, spk, \mathsf{report}) \rightarrow 1/0$. On input pp, the moderator secret key sk_{J}, a sender public key spk, and a report report, it outputs either 1 or 0.
- $\mathsf{Forge}(pp, pk_{\mathsf{J}}, spk, m) \rightarrow \sigma$. On input pp, the moderator public key pk_{J}, a sender public key spk and a message m, it outputs a "forged" franking signature σ.

Correctness: Given any pp generated by Setup, key pairs (spk, ssk) and (rpk_j, rsk_j) from SKGen and RKGen, and the key pair $(pk_{\mathsf{J}}, sk_{\mathsf{J}})$ from JKGen, we require that for any $\mathcal{S}$ contains rpk_j, if $ct \leftarrow \mathsf{Encrypt}(pp, pk_{\mathsf{J}}, spk, ssk, \mathcal{S}, m)$, then $(m, \sigma) \leftarrow \mathsf{Decrypt}(pp, pk_{\mathsf{J}}, spk, rsk_j, ct)$, and $(1, \mathsf{report}) \leftarrow \mathsf{Report}(pp, pk_{\mathsf{J}}, spk, m, \sigma)$, $1 \leftarrow \mathsf{Judge}(pp, sk_{\mathsf{J}}, spk, \mathsf{report})$.

Note that the Forge algorithm is not designed for normal use during data sharing, but instead serves a theoretical purpose to capture deniability in post-compromise settings. In scenarios where a message and its associated franking signature are disclosed by a malicious receiver, it is crucial to ensure that external parties cannot irrefutably attribute the message to the original sender. The Forge algorithm models the ability of an authorized receiver to simulate a valid-looking franking signature without access to the sender's secret key, such that the forged output is computationally indistinguishable from a genuine one. This ensures that only the moderator can verify reported messages, while external parties cannot confirm authorship, thereby preserving plausible deniability.

C. Security Model

In MBAC, both data senders and receivers are untrustworthy, as senders could impersonate others to disseminate fake data or bypass content moderation, while receivers might attempt to extract sensitive information from unauthorized ciphertexts or submit false reports to frame innocent senders. We then define the security model of FBME with four security notions: privacy, authenticity, accountability, and deniability.

- *Privacy:* FBME guarantees both confidentiality and anonymity for the sender and receiver. Only intended

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:12:53 UTC from IEEE Xplore. Restrictions apply.

$\mathbf{Game}^{\text{Privacy}}_{\text{FBME},\mathcal{A},n}(\lambda)$:
$pp \leftarrow \mathsf{Setup}(\lambda, n)$
$(pk_{\mathsf{J}}, sk_{\mathsf{J}}) \leftarrow \mathsf{JKGen}(pp)$
$\mathsf{U} = \emptyset; \mathsf{U_s} = \emptyset; \mathsf{U_r} = \emptyset; \mathsf{U_{cor}} = \emptyset; \mathsf{Q_{ct}} = \emptyset$
$\mathsf{Init}(\mathsf{U}, \mathsf{U_s}, \mathsf{U_r}, n)$
$(m_0, m_1, \mathcal{S}_0, \mathcal{S}_1, spk_0, spk_1) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{Cor_1}}, \mathcal{O}_{\mathsf{Cor_2}}, \mathcal{O}_{\mathsf{Dec}}}(pp, \mathsf{U}, ct)$
If $|m_0| \neq |m_1| \vee (|\mathcal{S}_0| \neq |\mathcal{S}_1|)$: Return 0
$\mathcal{S}_0 \Delta \mathcal{S}_1 \leftarrow (\mathcal{S}_0 \setminus \mathcal{S}_1) \cup (\mathcal{S}_1 \setminus \mathcal{S}_0)$
If $\mathsf{U_{cor}} \cap (\mathcal{S}_0 \Delta \mathcal{S}_1) \neq \emptyset$: Return 0
If $\mathsf{U_{cor}} \cap (\mathcal{S}_0 \cap \mathcal{S}_1) \neq \emptyset \wedge (m_0 \neq m_1)$: Return 0
$b \in \{0, 1\}$
$ct^* = \mathsf{Encrypt}(pp, pk_{\mathsf{J}}, spk_b, ssk_b, \mathcal{S}_b, m_b)$
$b' \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{Cor_1}}, \mathcal{O}_{\mathsf{Cor_2}}, \mathcal{O}_{\mathsf{Dec}}}(pp, \mathsf{U}, ct)$
If $\mathsf{U_{cor}} \cap (\mathcal{S}_0 \Delta \mathcal{S}_1) \neq \emptyset$: Return 0
If $ct^* \in \mathsf{Q_{ct}}$:
  If $\mathsf{U_{cor}} \cap (\mathcal{S}_0 \cap \mathcal{S}_1) \neq \emptyset \wedge (m_0 \neq m_1)$: Return 0
Return $b' = b$

$\mathbf{Game}^{\text{Auth}}_{\text{FBME},\mathcal{A},n}(\lambda)$:
$pp \leftarrow \mathsf{Setup}(\lambda, n)$
$(pk_{\mathsf{J}}, sk_{\mathsf{J}}) \leftarrow \mathsf{JKGen}(pp)$
$\mathsf{U} = \emptyset; \mathsf{U_s} = \emptyset; \mathsf{U_r} = \emptyset; \mathsf{U_{cor}} = \emptyset; \mathsf{Q_{ct}} = \emptyset$
$\mathsf{Init}(\mathsf{U}, \mathsf{U_s}, \mathsf{U_r}, n)$
$(spk^*, rpk^*, ct^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{Cor_1}}, \mathcal{O}_{\mathsf{Cor_2}}, \mathcal{O}_{\mathsf{Enc_1}}}(pp, \mathsf{U})$
If $\mathsf{Decrypt}(pp, pk_{\mathsf{J}}, spk^*, rsk^*, ct^*) \neq \perp$:
  If $spk^* \notin \mathsf{U_{cor}} \wedge ct^* \notin \mathsf{Q_{ct}}$: Return 1
Return 0

$\mathbf{Game}^{\text{Den}}_{\text{FBME},\mathcal{A},n}(\lambda)$:
$b \in \{0, 1\}$, $pp \leftarrow \mathsf{Setup}(\lambda, n)$
$(pk_{\mathsf{J}}, sk_{\mathsf{J}}) \leftarrow \mathsf{JKGen}(pp)$
$\mathsf{U} = \emptyset; \mathsf{U_s} = \emptyset; \mathsf{U_r} = \emptyset; \mathsf{U_{cor}} = \emptyset$
$\mathsf{Init}(\mathsf{U}, \mathsf{U_s}, \mathsf{U_r}, n)$
$b' \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{Cor_1}}, \mathcal{O}_{\mathsf{Cor_2}}, \mathcal{O}_{\mathsf{Forge}}}(pp, pk_{\mathsf{J}}, \mathsf{U})$
Return $b' = b$

$\mathbf{Game}^{\text{s-bind}}_{\text{FBME},\mathcal{A},n}(\lambda)$:
$pp \leftarrow \mathsf{Setup}(\lambda, n)$
$(pk_{\mathsf{J}}, sk_{\mathsf{J}}) \leftarrow \mathsf{JKGen}(pp)$
$\mathsf{U} = \emptyset; \mathsf{U_s} = \emptyset; \mathsf{U_r} = \emptyset; \mathsf{U_{cor}} = \emptyset$
$\mathsf{Init}(\mathsf{U}, \mathsf{U_s}, \mathsf{U_r}, n)$
$(spk^*, rpk^*, ct^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{Cor_1}}, \mathcal{O}_{\mathsf{Cor_2}}, \mathcal{O}_{\mathsf{Enc_2}}, \mathcal{O}_{\mathsf{Judge}}}(pp, pk_{\mathsf{J}}, \mathsf{U})$
If $rpk^* \in \mathsf{U_{cor}}$: Return 0
$(m^*, \sigma^*) \leftarrow \mathsf{Decrypt}(pp, pk_{\mathsf{J}}, spk^*, rsk^*, ct^*)$
$(b_1, \mathsf{report}^*) \leftarrow \mathsf{Report}(pp, pk_{\mathsf{J}}, spk^*, m^*, \sigma^*)$
$b_2 \leftarrow \mathsf{Judge}(pp, sk_{\mathsf{J}}, spk^*, \mathsf{report}^*)$
Return $b_1 \wedge \neg b_2$

$\mathbf{Game}^{\text{r-bind}}_{\text{FBME},\mathcal{A},n}(\lambda)$:
$pp \leftarrow \mathsf{Setup}(\lambda, n)$
$(pk_{\mathsf{J}}, sk_{\mathsf{J}}) \leftarrow \mathsf{JKGen}(pp)$
$\mathsf{U} = \emptyset; \mathsf{U_s} = \emptyset; \mathsf{U_r} = \emptyset; \mathsf{U_{cor}} = \emptyset; \mathsf{Q_{sig}} = \emptyset$
$\mathsf{Init}(\mathsf{U}, \mathsf{U_s}, \mathsf{U_r}, n)$
$(spk^*, rpk^*, ct^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{Cor_1}}, \mathcal{O}_{\mathsf{Cor_2}}, \mathcal{O}_{\mathsf{Enc_3}}, \mathcal{O}_{\mathsf{Judge}}}(pp, pk_{\mathsf{J}}, \mathsf{U})$
$(m^*, \sigma^*) \leftarrow \mathsf{Decrypt}(pp, pk_{\mathsf{J}}, spk^*, rsk^*, ct^*)$
$(1, \mathsf{report}^*) \leftarrow \mathsf{Report}(pp, pk_{\mathsf{J}}, spk^*, m^*, \sigma^*)$
If $\mathsf{Judge}(pp, sk_{\mathsf{J}}, spk^*, \mathsf{report}^*) = 1$:
  If $(spk^* \notin \mathsf{U_{cor}}) \wedge (\nexists (spk^*, \mathcal{S}, m^*) \in \mathsf{Q_{sig}})$: Return 1
Return 0

Fig. 4. Games for privacy, authenticity, sender binding, receiver binding and deniability of FBME.

receivers can decrypt the ciphertext. Sender anonymity ensures that unauthorized entities cannot infer the sender's identity from the ciphertext, while receiver anonymity ensures that even authorized receivers cannot learn who else is in the recipient set. A FBME scheme satisfies privacy if for any PPT adversary $\mathcal{A}$, its advantage $\mathsf{Adv}^{\text{Privacy}}_{\text{FBME},\mathcal{A}} = |\Pr[\mathbf{Game}^{\text{Privacy}}_{\text{FBME},\mathcal{A},n}(\lambda) = 1] - \frac{1}{2}|$ is negligible.

- *Authenticity:* FBME ensures that only a sender with a secret key can produce a valid ciphertext. A FBME scheme satisfies authenticity if for any PPT adversary $\mathcal{A}$, its advantage $\mathsf{Adv}^{\text{Auth}}_{\text{FBME},\mathcal{A}} = \Pr[\mathbf{Game}^{\text{Auth}}_{\text{FBME},\mathcal{A},n}(\lambda) = 1]$ is negligible.
- *Accountability:* Following the terminology in AMF [10], accountability is characterized by two properties: sender binding and receiver binding. The former ensures that a malicious sender cannot deceive receivers into accepting unreportable data, while the latter prevents receivers from tricking the moderator into framing an honest sender. A FBME scheme satisfies accountability if for any PPT adversary $\mathcal{A}$, its advantage $\mathsf{Adv}^{\text{s-bind}}_{\text{FBME},\mathcal{A}} = \Pr[\mathbf{Game}^{\text{s-bind}}_{\text{FBME},\mathcal{A},n}(\lambda) = 1]$ and $\mathsf{Adv}^{\text{r-bind}}_{\text{FBME},\mathcal{A}} = \Pr[\mathbf{Game}^{\text{r-bind}}_{\text{FBME},\mathcal{A},n}(\lambda) = 1]$ are negligible.
- *Deniability:* FBME ensures that the receivers without the sender secret key can create a franking signature indistinguishable from a legitimate one for any non-participating parties (i.e., no access to sender or moderator secret keys). Intuitively, this enables the sender to assert that a message was generated by any non-participating party. A FBME scheme satisfies deniability if for any PPT adversary $\mathcal{A}$, its advantage $\mathsf{Adv}^{\text{Den}}_{\text{FBME},\mathcal{A}} = |\Pr[\mathbf{Game}^{\text{Den}}_{\text{FBME},\mathcal{A},n}(\lambda) = 1] - \frac{1}{2}|$ is negligible.

The security of FBME is formalized through games between an adversary $\mathcal{A}$ and a challenger $\mathcal{C}$. To simplify the description of these security games, we define an initialization method $\mathsf{Init}(\mathsf{U}, \mathsf{U_s}, \mathsf{U_r}, n)$ that initializes the system's key pairs as follows: For each sender s, the method generates $(spk, ssk) \leftarrow \mathsf{SKGen}(pp)$, and updates $\mathsf{U} \leftarrow \mathsf{U} \cup \{spk\}$ and $\mathsf{U_s} \leftarrow \mathsf{U_s} \cup \{(spk, ssk)\}$. For each receiver $\mathsf{r}_j \in \mathcal{U}$, the method generates $(rpk_j, rsk_j) \leftarrow \mathsf{RKGen}(pp)$, and updates $\mathsf{U} \leftarrow \mathsf{U} \cup \{rpk_j\}$ and $\mathsf{U_r} \leftarrow \mathsf{U_r} \cup \{(rpk_j, rsk_j)\}$. The formal security definitions and the oracles for the FBME security games are provided in Figs. 4 and 5, respectively.

$\mathcal{O}_{\mathsf{Cor_1}}(spk)$:
$\mathsf{U_{cor}} \leftarrow \mathsf{U_{cor}} \cup \{spk\}$
Return ssk s.t $(spk, ssk) \in \mathsf{U_s}$

$\mathcal{O}_{\mathsf{Enc_1}}(pp, spk, \mathcal{S}, m)$:
$ssk \leftarrow \mathsf{U_s}$ s.t $(spk, ssk) \in \mathsf{U_s}$
$ct \leftarrow \mathsf{Encrypt}(pp, pk_{\mathsf{J}}, spk, ssk, \mathcal{S}, m)$
$\mathsf{Q_{ct}} \leftarrow \mathsf{Q_{ct}} \cup \{ct\}$
Return ct

$\mathcal{O}_{\mathsf{Enc_2}}(pp, spk, \mathcal{S}, m)$:
$ssk \leftarrow \mathsf{U_s}$ s.t $(spk, ssk) \in \mathsf{U_s}$
Return $\mathsf{Encrypt}(pp, pk_{\mathsf{J}}, spk, ssk, \mathcal{S}, m)$

$\mathcal{O}_{\mathsf{Enc_3}}(pp, spk, \mathcal{S}, m)$:
$\mathsf{Q_{sig}} \leftarrow \mathsf{Q_{sig}} \cup \{(spk, \mathcal{S}, m)\}$
$ssk \leftarrow \mathsf{U_s}$ s.t $(spk, ssk) \in \mathsf{U_s}$
Return $\mathsf{Encrypt}(pp, pk_{\mathsf{J}}, spk, ssk, \mathcal{S}, m)$

$\mathcal{O}_{\mathsf{Cor_2}}(rpk_j)$:
$\mathsf{U_{cor}} \leftarrow \mathsf{U_{cor}} \cup \{rpk_j\}$
Return rsk_j s.t $(rpk_j, rsk_j) \in \mathsf{U_r}$

$\mathcal{O}_{\mathsf{Dec}}(pp, spk, rpk_j, ct)$:
$\mathsf{Q_{ct}} \leftarrow \mathsf{Q_{ct}} \cup \{ct\}$
$rsk_j \leftarrow \mathsf{U_r}$ s.t $(rpk_j, rsk_j) \in \mathsf{U_r}$
Return $\mathsf{Decrypt}(pp, pk_{\mathsf{J}}, spk, rsk_j, ct)$

$\mathcal{O}_{\mathsf{Forge}}(pp, pk_{\mathsf{J}}, spk, m)$:
$ssk \leftarrow \mathsf{U_s}$ s.t $(spk, ssk) \in \mathsf{U_s}$
$ct \leftarrow \mathsf{Encrypt}(pp, pk_{\mathsf{J}}, spk, ssk, \mathcal{S}, m)$
$(m, \sigma_0) \leftarrow \mathsf{Decrypt}(pp, pk_{\mathsf{J}}, spk, rsk_j, ct)$
$\sigma_1 \leftarrow \mathsf{Forge}(pp, pk_{\mathsf{J}}, spk, m)$
Return σ_b

$\mathcal{O}_{\mathsf{Judge}}(pp, spk, \mathsf{report})$:
Return $\mathsf{Judge}(pp, sk_{\mathsf{J}}, spk, \mathsf{report})$

Fig. 5. Oracles for games of FBME.

V. MBAC Construction

A. Technical Overview

FBME adopts a sign-then-encrypt mechanism to realize efficient content-moderated bilateral access control over anonymous multiple receivers. It addresses two key aspects: enforcing bilateral access with privacy and authenticity, and enabling content moderation with accountability and deniability. The high-level description is outlined below.

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:12:53 UTC from IEEE Xplore. Restrictions apply.

- *Privacy and authenticity:* To address the challenge of preserving receiver anonymity in multi-receiver bilateral access control, FBME employs two core strategies. First, FBME generates an asymmetric franking signature solely on the data m, avoiding the inclusion of encapsulated keys linked to each target recipient as required in AGMF [12]. This ensures that even if an authorized receiver decrypts and retrieves the franking signature, it reveals nothing about other privileged recipients. Second, at the encryption layer, FBME encrypts both the data and its franking signature for intended receivers using the RR-KDPKE [27], which provides key-privacy to ensure that the ciphertext leaks no information about the receiver's public key. Specifically, for each target recipient $j \in \mathcal{S}$, FBME derives two message keys: $K_{\mathsf{r},j}$ from the receiver's public key, and $K_{\mathsf{s},j}$ from both the receiver's public key and the sender's secret key under the CBDH assumption. These keys form ciphertext components that are bound together via a one-time signature, which achieves sender identification while maintaining data confidentiality with CCA security and preserving receiver anonymity.
- *Accountability and deniability:* Another key requirement of MBAC is to enable verifiable reporting between receivers and the moderator, while preserving the sender's deniability. To achieve this, FBME employs a NIZK proof system based on Sigma protocols with Fiat-Shamir transform to construct a NIZK proof π for a carefully designed relation $\mathcal{R}$ involving the sender secret key, and incorporates an encapsulated ciphertext c and key k_{J} specifically for the moderator into the franking signature. During moderation, authorized receivers can verify that the franking signature intended for the moderator is well-formed by validating the NIZK proof π with respect to $\mathcal{R}$, while the moderator additionally uses their private key to validate c and k_{J}. This design ensures that only the moderator can fully validate the franking signature, effectively balancing accountability and deniability.

B. Concrete Construction

We describe the detailed workflow of MBAC based on FBME = (Setup, SKGen, RKGen, JKGen, Encrypt, Decrypt, Report, Judge, Forge), which operates through four main phases. Initially, the system initialization phase establishes necessary system parameters and generates keys for participants. Then, the sender generates a franked ciphertext for the intended receivers during the data publishing phase, and the receivers decrypt the ciphertext while verifying sender authenticity during the data receiving phase. Finally, in the content moderation phase, receivers can generate and submit reports for malicious content to the service provider, who verifies the report's validity and holds the sender accountable if necessary. Fig. 6 illustrates an overview of the workflow.

System Initialization: The system starts by running the Setup algorithm to generate public parameters. Data senders, receivers, and the moderator respectively generate key pairs via SKGen, RKGen, and JKGen algorithms, and register their public keys with the CA for certification.

- $\mathsf{Setup}(\lambda, n)$. It picks a group $\mathbb{G}$ of prime order p with generators $g_1, g_2 \in \mathbb{G}$ and a bilinear pairing $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$. Then, it chooses two collusion-resistant hash functions $\mathsf{H}_1 : \{0,1\}^* \to \mathbb{Z}_p^*$, $\mathsf{H}_2 : \mathbb{Z}_p^* \to \mathbb{G}$, two collusion-resistant key derivation functions [30] $\mathsf{H} : \mathbb{G} \to \{0,1\}^\eta$, $\hat{\mathsf{H}} : \mathbb{G}_T \to \{0,1\}^\eta$ for some $\eta \in \mathsf{poly}(\lambda)$, both modeled as secure pseudorandom functions (PRF). It also selects a SUF one-time signature scheme [31] $\Sigma = (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vrf})$. The public parameter consists of $pp = (\mathbb{G}, p, e, g_1, g_2, \mathsf{H}_1, \mathsf{H}_2, \mathsf{H}, \hat{\mathsf{H}}, \Sigma)$, which implicitly includes the output of the RR-KDPKE.Setup algorithm.
- $\mathsf{SKGen}(pp)$. It picks random values $s, \theta_1, \theta_2 \in \mathbb{Z}_p^*$, computes $E_1 = g_1^{\theta_1} g_2^{\theta_2}, E_2 = g_1^s$, and sets the public key as $spk = (E_1, E_2)$ and the secret key as $ssk = (s, \theta_1, \theta_2)$.
- $\mathsf{RKGen}(pp)$. It first executes RR-KDPKE.KGen by selecting random values $x_{1,j}, x_{2,j}, y_{1,j}, y_{2,j} \in \mathbb{Z}_p^*$ and computing $D_{1,j} = g_1^{x_{1,j}} g_2^{x_{2,j}}$, $D_{2,j} = g_1^{y_{1,j}} g_2^{y_{2,j}}$. Then, it picks $\varphi_j \in \mathbb{Z}_p^*$, and computes $D_{3,j} = g_1^{\varphi_j}$. Finally, it sets the public key as $rpk_j = (D_{1,j}, D_{2,j}, D_{3,j})$ and the secret key as $rsk_j = (x_{1,j}, x_{2,j}, y_{1,j}, y_{2,j}, \varphi_j)$.
- $\mathsf{JKGen}(pp)$. It selects two random values $\varpi_1, \varpi_2 \in \mathbb{Z}_p^*$, and sets the public key as $pk_{\mathsf{J}} = g_1^{\varpi_1} g_2^{\varpi_2}$ and the secret key as $sk_{\mathsf{J}} = (\varpi_1, \varpi_2)$.

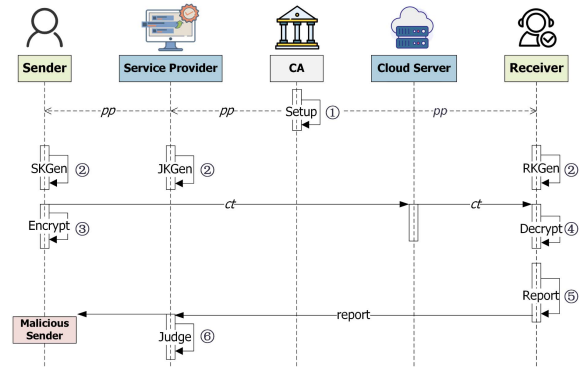


Fig. 6. MBAC workflow.

Data Publishing: For each data, the sender uses the Encrypt algorithm to generate a franked ciphertext for the target receivers. The franked ciphertext is then uploaded to the service provider, which outsources its storage to the cloud server.

- $\mathsf{Encrypt}(pp, pk_{\mathsf{J}}, spk, ssk, \mathcal{S}, m)$. Parse $ssk = (s, \theta_1, \theta_2)$. To encrypt the data m for a target receiver set $\mathcal{S} = \{rpk_{\phi_1}, rpk_{\phi_2}, \ldots, rpk_{\phi_\ell}\}$, the encryption algorithm proceeds in two steps, as shown in Fig. 7. In **Step 1**, it computes a franking signature σ on m, defined by the function Frank. In particular, the relation $\mathcal{R}$ consists of three sub-relations:

$$\mathcal{R}_s = \{((\theta_1, \theta_2), E_1) : E_1 = g_1^{\theta_1} g_2^{\theta_2}\},$$
$$\mathcal{R}_{c,k} = \{(r, (c, k_{\mathsf{J}}, pk_{\mathsf{J}})) : u_1 = g_1^r \wedge u_2 = g_2^r \wedge k_{\mathsf{J}} = pk_{\mathsf{J}}^r\},$$
$$\mathcal{R}_c^* = \{((r, r'), (u_1, u_2)) : u_1 = g_1^r, u_2 = g_1^{r'}\}.$$

$\mathcal{R}_s$ represents a relation that proves the validity of the sender's public/secret keys, $\mathcal{R}_{c,k}$ denotes a relation that proves (c, k_{J}) are generated using the same randomness r, and $\mathcal{R}_c^*$ denotes a relation that proves c is ill-formed, generated with $r^* = (r, r')$. By combining these sub-relations, we define the complete relation $\mathcal{R}$ as follows:

$$\mathcal{R} = \{((\theta_1, \theta_2, r, r^*), (E_1, pk_{\mathsf{J}}, c, k_{\mathsf{J}})) : (((\theta_1, \theta_2), E_1) \in \mathcal{R}_s \wedge (r, (c, k_{\mathsf{J}}, pk_{\mathsf{J}})) \in \mathcal{R}_{c,k}) \vee (r^*, c) \in \mathcal{R}_c^*\}.$$

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:12:53 UTC from IEEE Xplore. Restrictions apply.

- **Step 1:** Generate the franking signature on message m: $\sigma \leftarrow \mathsf{Frank}(pp, pk_{\mathsf{J}}, spk, ssk, m)$. It chooses $r \in \mathbb{Z}_p^*$, computes $u_1 = g_1^r, u_2 = g_2^r$, and sets the encapsulated ciphertext component as $c = (u_1, u_2)$. It then calculates the encapsulated key $k_{\mathsf{J}} = pk_{\mathsf{J}}^r$. Set statement $\mathcal{V} = (E_1, pk_{\mathsf{J}}, c, k_{\mathsf{J}})$ and witness $\mathcal{W} = (\theta_1, \theta_2, r, \perp)$. Using the NIZK proof system for a relation $\mathcal{R}$, it generates a NIZK proof $\pi \leftarrow \mathsf{NIZK.Prove}(m, \mathcal{V}, \mathcal{W})$, and sets the franking signature as $\sigma = (\pi, c, k_{\mathsf{J}})$.
- **Step 2:** Let the franked message be $m_{\mathsf{frank}} = m||\sigma$, which is then encrypted for each target receiver. The following is obtained from the RR-KDPKE.Enc algorithm by removing redundant ciphertext components from the overall ciphertext and adding a message key K_{s,ϕ_i} with the sender secret key to encapsulate the message ciphertext A_i for each $rpk_{\phi_i} \in \mathcal{S}$. Notably, (V_1, V_2) appears only once in the FBME ciphertext and this modification does not impact the security analysis.
  - It generates a key pair $(ak, vk) \leftarrow \Sigma.\mathsf{KGen}(\lambda)$. Then, it picks $w \in \mathbb{Z}_p^*$ and computes $V_1 = g_1^w, V_2 = g_2^w$ along with $\alpha = \mathsf{H}_1(V_1, V_2, vk)$.
  - For each $rpk_{\phi_i} = (D_{1,\phi_i}, D_{2,\phi_i}, D_{3,\phi_i}) \in \mathcal{S}$, it computes message keys $K_{\mathsf{r},\phi_i} = (D_{1,\phi_i} D_{2,\phi_i}^{\alpha})^w$ and $K_{\mathsf{s},\phi_i} = e(\mathsf{H}_2(\alpha)^s, D_{3,\phi_i})$.
  - Then, it calculates a message ciphertext $A_i = m_{\mathsf{frank}} \oplus \mathsf{H}(K_{\mathsf{r},\phi_i}) \oplus \hat{\mathsf{H}}(K_{\mathsf{s},\phi_i})$. It sets $C = (V_1, V_2, \{A_{\tau(i)}\}_{i \in [1,\ell]})$, where $\tau : \{1, 2, \ldots, \ell\} \rightarrow \{1, 2, \ldots, \ell\}$ is a random permutation.

  Finally, it computes the signature $\varsigma \leftarrow \Sigma.\mathsf{Sign}(ak, C)$, and sets the franked ciphertext as $ct = (vk, C, \varsigma)$.

Fig. 7. The encryption algorithm of FBME.

- The first part of $\mathcal{R}$ (i.e., $((\theta_1, \theta_2), E_1) \in \mathcal{R}_s \wedge (r, (c, k_{\mathsf{J}}, pk_{\mathsf{J}})) \in \mathcal{R}_{c,k}$) consist of two sub-parts: 1) $(\theta_1, \theta_2, E_1) \in \mathcal{R}_s$ guarantees the sender's authentication for the moderator; 2) $(r, (c, k_{\mathsf{J}}, pk_{\mathsf{J}})) \in \mathcal{R}_{c,k}$ ensures that the ciphertext component c and the encapsulated key k_{J} for the moderator are well-formed, and convince the receiver that (c, k_{J}) can be successfully verified by the moderator.
- The second part of $\mathcal{R}$ (i.e., $(r^*, c) \in \mathcal{R}_c^*$) is specifically designed to ensure deniability. In other words, it is prepared for the forgeries to create a valid NIZK proof without access to the sender's secret key, simultaneously preventing the formation of valid franking signatures.

The relation $\mathcal{R}$ uses an "OR" operation to combine the two parts, enabling either the sender or the forgers to produce a valid NIZK proof. Specifically, Okamoto's Sigma protocol [32] is used for $\mathcal{R}_s$, Chaum-Pedersen's Sigma protocol [33] is applied to $\mathcal{R}_{c,k}$, and Schnorr's Sigma protocol [34] is utilized for $\mathcal{R}_c^*$. Thus, we can obtain a NIZK proof system for $\mathcal{R}$ by using the Fiat-Shamir transform [35] with the composition operations of Sigma protocols [36]. In **Step 2**, it encrypts the franked message m_{frank} (i.e., the concatenation of m and σ) to produce the final franked ciphertext ct, where the ciphertext encapsulation is dependent on both K_{r,ϕ_i} and K_{s,ϕ_i}.

Data Receiving: The receiver runs the Decrypt algorithm to authenticate the sender, and decrypt the franked ciphertext to recover the original plaintext.

- $\mathsf{Decrypt}(pp, pk_{\mathsf{J}}, spk, rsk_j, ct)$. Parse $ct = (vk, C = (V_1, V_2, \{A_\imath\}_{\imath \in [1,\ell]}), \varsigma)$, and $spk = (E_1, E_2)$, $rsk_j = (x_{1,j}, x_{2,j}, y_{1,j}, y_{2,j}, \varphi_j)$. The receiver retrieves m from ct as follows.
  - If $\Sigma.\mathsf{Vrf}(vk, C, \varsigma) = 0$, it returns $\perp$; otherwise, following the RR-KDPKE.Dec algorithm, it computes $\alpha = \mathsf{H}_1(V_1, V_2, vk)$ and $K'_{\mathsf{r},j} = V_1^{x_{1,j}+\alpha y_{1,j}} \cdot V_2^{x_{2,j}+\alpha y_{2,j}}$. Then, it derives $K'_{\mathsf{s},j} = e(\mathsf{H}_2(\alpha)^{\varphi_j}, E_2)$ with spk.
  - For each $A_\imath$, it computes $m_{\mathsf{frank}} = A_\imath \oplus \mathsf{H}(K'_{\mathsf{r},j}) \oplus \hat{\mathsf{H}}(K'_{\mathsf{s},j})$ where $\phi_\imath = j$. If m_{frank} can be parsed as $m||\sigma$, it returns m; otherwise, it outputs $\perp$.

Correctness: If $rpk_j \in \mathcal{S}$, there exists an index $\phi_\imath = j$ such that m_{frank} can be recovered from the corresponding ciphertext component $A_\imath$ associated with $rsk_{\phi_\imath}$.

$$
\begin{aligned}
m_{\mathsf{frank}} &= A_\imath \oplus \mathsf{H}(K'_{\mathsf{r},j}) \oplus \hat{\mathsf{H}}(K'_{\mathsf{s},j}) \\
&= m_{\mathsf{frank}} \oplus \mathsf{H}(K_{\mathsf{r},\phi_\imath}) \oplus \hat{\mathsf{H}}(K_{\mathsf{s},\phi_\imath}) \oplus \mathsf{H}(K'_{\mathsf{r},j}) \oplus \hat{\mathsf{H}}(K'_{\mathsf{s},j}) \\
&= m_{\mathsf{frank}} \oplus \mathsf{H}((D_{1,\phi_\imath} D_{2,\phi_\imath}^{\alpha})^w) \oplus \hat{\mathsf{H}}(e(\mathsf{H}_2(\alpha)^s, D_{3,\phi_\imath})) \\
&\oplus \mathsf{H}(V_1^{x_{1,j}+\alpha y_{1,j}} V_2^{x_{2,j}+\alpha y_{2,j}}) \oplus \hat{\mathsf{H}}(e(\mathsf{H}_2(\alpha)^{\varphi_j}, E_2)) \\
&= m_{\mathsf{frank}} \oplus \mathsf{H}((g_1^{x_{1,\phi_\imath}} g_2^{x_{2,\phi_\imath}} (g_1^{y_{1,\phi_\imath}} g_2^{y_{2,\phi_\imath}})^{\alpha})^w) \\
&\oplus \hat{\mathsf{H}}(e(\mathsf{H}_2(\alpha)^s, g_1^{\varphi_{\phi_\imath}})) \oplus \mathsf{H}(g_1^{w(x_{1,j}+\alpha y_{1,j})} g_2^{w(x_{2,j}+\alpha y_{2,j})}) \\
&\oplus \hat{\mathsf{H}}(e(\mathsf{H}_2(\alpha)^{\varphi_j}, g_1^s)).
\end{aligned}
$$

Content Moderation: In the content moderation phase, if a receiver suspects that the content is malicious, they execute the Report algorithm to generate a report, which is then sent to the service provider for further judgment.

- $\mathsf{Report}(pp, pk_{\mathsf{J}}, spk, m, \sigma)$. If m is deemed malicious, the receiver proceeds with the following verification. Parse $\sigma = (\pi, c, k_{\mathsf{J}})$. It sets statement $\mathcal{V} = (E_1, pk_{\mathsf{J}}, c, k_{\mathsf{J}})$. If $\mathsf{NIZK.Verify}(m, \mathcal{V}, \pi) = 0$, it returns 0; otherwise, it outputs 1. If the verification succeeds, it also sets $\mathsf{report} = (m, \sigma)$.

Upon receiving a report from a receiver, the service provider runs the Judge algorithm to verify the report and hold the sender accountable for malicious content. It first verifies the NIZK proof, then uses the moderator secret key to check if the encapsulated key k_{J} matches the corresponding key k'_{J} derived from the ciphertext. In this way, the service provider can impose appropriate penalties, effectively discouraging the sender from distributing harmful data.

- $\mathsf{Judge}(pp, sk_{\mathsf{J}}, spk, \mathsf{report})$. Parse $spk = (E_1, E_2)$, $\mathsf{report} = (m, \sigma)$, and $\sigma = (\pi, c, k_{\mathsf{J}})$. It then sets the statement $\mathcal{V} = (E_1, pk_{\mathsf{J}}, c, k_{\mathsf{J}})$. If $\mathsf{NIZK.Verify}(m, \mathcal{V}, \pi) = 0$, it return 0. Otherwise, it computes $k'_{\mathsf{J}} = u_1^{\varpi_1} u_2^{\varpi_2}$ and if $k'_{\mathsf{J}} \neq k_{\mathsf{J}}$, return 0.

In addition, we equip FBME with the Forge algorithm to forge convincing-looking franking signatures that are indistinguishable from a legitimate one.

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:12:53 UTC from IEEE Xplore. Restrictions apply.

- $\mathsf{Forge}(pp, pk_{\mathsf{J}}, spk, m)$. Parse $spk = (E_1, E_2)$. It first chooses $r^* = (r, r') \in \mathbb{Z}_p^{*2}$, computes $u_1 = g_1^r, u_2 = g_1^{r'}$, and sets the encapsulated ciphertext component as $c^* = (u_1, u_2)$. Next, it calculates an encapsulated key $k_{\mathsf{J}} = pk_{\mathsf{J}}^r$. Following this, it sets witness $\mathcal{W} = (\perp, \perp, \perp, r^*)$, and statement $\mathcal{V} = (E_1, pk_{\mathsf{J}}, c, k_{\mathsf{J}})$. Then, it employs the NIZK proof system for relation $\mathcal{R}$ to generate a NIZK proof $\pi \leftarrow \mathsf{NIZK.Prove}(m, \mathcal{V}, \mathcal{W})$. Finally, it sets the "forged" franking signature as $\sigma = (\pi, c, k_{\mathsf{J}})$.

VI. Security Proofs and Analysis

A. Security Proofs of FBME

Theorem 1: FBME achieves privacy under the following assumptions: 1) RR-KDPKE is AI-CCA secure, WR-CCA secure, and key-less reproducible; 2) The one-time signature scheme Σ is SUF and UL; 3) The key derivation function $\hat{\mathsf{H}}$ is a secure PRF.

Proof: We prove the theorem via a standard game-hopping approach. The adversary $\mathcal{A}$ initially receives an encryption of $(m_0, \mathcal{S}_0, spk_0)$, which is gradually transformed into an encryption of $(m_1, \mathcal{S}_1, spk_1)$ through a series of hybrid games. Since $|\mathcal{S}_0| = |\mathcal{S}_1|$, it follows that $|\mathcal{S}_0 \setminus \mathcal{S}_1| = |\mathcal{S}_1 \setminus \mathcal{S}_0| = |\mathcal{S}_b| - |\mathcal{S}_0 \cap \mathcal{S}_1|$ for $b \in \{0, 1\}$.

- $Game_0^{\mathsf{Real}}$: This represents the real game when the challenger $\mathcal{C}$ selects the bit $b = 0$. In the first phase, $\mathcal{A}$ adaptively query secret keys ssk and $rsk_j \in \mathcal{U}$. $\mathcal{A}$ can query the decryption oracle with (spk, rpk_j, ct), which is answered using the corresponding keys spk, rsk_j. In the challenge phase, $\mathcal{A}$ submits messages m_0, m_1, receiver sets $\mathcal{S}_0, \mathcal{S}_1 \subset \mathcal{U}$, and two sender public keys spk_0, spk_1 such that $|m_0| = |m_1|$ and $|\mathcal{S}_0| = |\mathcal{S}_1| = \ell$. Then, $\mathcal{C}$ obtains a key pair of one-time signature $(ak^*, vk^*) \leftarrow \Sigma.\mathsf{KGen}(\lambda)$, parses the indices of $\mathcal{S}_0$ as $\{\vartheta_1, \vartheta_2, \ldots, \vartheta_\ell\}$ and returns the challenge ciphertext $ct^* = (vk^*, C = (V_1, V_2, \{A_{\tau(i)}\}_{i\in[1,\ell]}), \varsigma)$, where V_1, V_2 are shared components across $\{A_{\tau(i)}\}_{i\in[1,\ell]}$ from the same randomness $w \in \mathbb{Z}_p^*$, and τ is a random permutation. Specifically, it first computes $\bar{A} \leftarrow \mathsf{RR\text{-}KDPKE.Enc}(rpk_{\vartheta_1}, m_0; w)$, and generates $\bar{A}_i \leftarrow \mathsf{Rep}(\bar{A}, m_0, rpk_{\vartheta_i}, rsk_{\vartheta_i})$ for each $i \in [2, \ell]$. Then, $\mathcal{C}$ computes $K_{\mathsf{s},\vartheta_i}$ with ssk_0 by following $\mathsf{Encrypt}$ algorithm and obtains $A_i = \bar{A}_i \oplus \hat{\mathsf{H}}(K_{\mathsf{s},\vartheta_i})$ for each $i \in [1, \ell]$. In the second phase, $\mathcal{A}$ may issue decryption queries and key queries with any public keys such that $\mathcal{U} \setminus (\mathcal{S}_0 \triangle \mathcal{S}_1)$. Eventually, $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$, where $\mathcal{E}_0^{\mathsf{Real}}$ denotes the event that $b' = 0$.
- $Game_0$: Same as $\mathbf{Game}_0^{\mathsf{Real}}$ except that $\mathcal{C}$ rejects any post-challenge decryption queries (spk, rpk_j, ct) if ct includes the verification key vk^* used in the challenge phase. Let $\mathcal{E}_0$ be the event where $\mathcal{A}$ outputs $b' = 0$ in $\mathbf{Game}_0$.
- $Game_\kappa$ $(\kappa \in [1, \ell])$: Given two challenge sets $\mathcal{S}_0, \mathcal{S}_1$ from $\mathcal{A}$, $\mathcal{C}$ defines $\xi = |\mathcal{S}_0 \cap \mathcal{S}_1|$. It reorders $\mathcal{S}_0$ and $\mathcal{S}_1$ with ordered indices $\{\vartheta_1, \ldots, \vartheta_\xi, \vartheta_{\xi+1}, \ldots, \vartheta_\ell\}$ and $\{\varrho_1, \ldots, \varrho_\xi, \varrho_{\xi+1}, \ldots, \varrho_\ell\}$ respectively, in such a way that $\vartheta_i = \varrho_i$ for each $i \in [1, \xi]$ and $\vartheta_i \neq \varrho_i$ if $i > \xi$. Then, $\mathcal{C}$ generates the challenge ciphertext as below:
  - For $i \in [1, \xi]$, set $\bar{A}_i = \mathsf{Rep}(\bar{A}, m_1, rpk_{\vartheta_i}, rsk_{\vartheta_i})$ if $i \leq \kappa$ and $\bar{A}_i = \mathsf{Rep}(\bar{A}, m_0, rpk_{\vartheta_i}, rsk_{\vartheta_i})$ if $i > \kappa$.
  - For $i \in [\xi + 1, \ell]$, set $\bar{A}_i = \mathsf{Rep}(\bar{A}, m_1, rpk_{\varrho_i}, rsk_{\varrho_i})$ if $i \leq \kappa$ and $\bar{A}_i = \mathsf{Rep}(\bar{A}, m_0, rpk_{\vartheta_i}, rsk_{\vartheta_i})$ if $i > \kappa$.

  $\mathcal{C}$ continues the remaining computation according to the $\mathsf{Encrypt}$ algorithm, and returns $ct^* = (vk^*, C = (V_1, V_2, \{A_{\tau(i)}\}_{i\in[1,\ell]}), \varsigma)$ to $\mathcal{A}$. Then, $\mathcal{C}$ handles the second phase following the prior games. Let $\mathcal{E}_\kappa$ denote the event that $\mathcal{A}$ outputs $b' = 0$ at the end of $\mathbf{Game}_\kappa$.$\mathbf{Game}'_\ell$: Identical to $\mathbf{Game}_\ell$ except the ciphertext is generated using spk_1 instead of spk_0.$\mathbf{Game}_\ell^{\mathsf{Real}}$: Same as $\mathbf{Game}'_\ell$ except that $\mathcal{C}$ does not reject ciphertexts containing the verification key vk^* when handling decryption queries. $\mathbf{Game}_\ell^{\mathsf{Real}}$ simulates the real game when the $\mathcal{C}$'s bit is $b = 1$. Let $\mathcal{E}_\ell^{\mathsf{Real}}$ be the event where $\mathcal{A}$ outputs $b' = 0$ at the conclusion of $\mathbf{Game}_\ell^{\mathsf{Real}}$.

Analysis: The goal is to show that each transition between adjacent games is computationally indistinguishable (denoted by $\approx$), such that $\mathcal{A}$ cannot distinguish between the initial and final games. The transitions are justified as follows:

- $Game_0^{\mathsf{Real}} \approx Game_0$: Their indistinguishability follows from the SUF property of the one-time signature, ensuring no adversary can forge a valid ciphertext under vk^*.
- $Game_{\kappa-1} \approx \mathbf{Game}_\kappa$ for all $\kappa \in [1, \ell]$: The transition from $\mathbf{Game}_0$ to $\mathbf{Game}_\ell$ changes one ciphertext component at a time from an encryption of m_0 to an encryption of m_1, either on common recipients $(i \leq \xi)$ or differing recipients $(i > \xi)$. The indistinguishability between each pair of adjacent games $\mathbf{Game}_{\kappa-1}$ and $\mathbf{Game}_\kappa$ is proven in Lemmas 1 and 2, under the AI-CCA and WR-CCA security of the underlying RR-KDPKE.
- $Game_\ell \approx Game'_\ell$: This transition swaps spk_0 with spk_1 in generating the challenge ciphertext, with the same message and recipients. Since the one-time signature verification key vk^* is unlinkable and the encapsulated components A_i are derived via a PRF from ssk_b, the ciphertext reveals no information about the sender's identity. Thus, this transition is indistinguishable under the unlinkability of Σ and the PRF security of $\hat{\mathsf{H}}$.
- $Game'_\ell \approx Game_\ell^{Real}$: This final transition re-enables decryption queries under vk^* and is again bounded by the SUF property of Σ.

Combining the above transitions, we obtain the following indistinguishability chain:

$$\begin{aligned}
&\mathbf{Game}_\mathbf{0}^{\mathsf{Real}}(\mathbf{spk_0}, \mathcal{S}_\mathbf{0}, \mathbf{m_0}) \approx \mathbf{Game_0} \approx \cdots \\
&\approx \mathbf{Game}_\ell(spk_0, \mathcal{S}_1, m_1) \approx \mathbf{Game}'_\ell(spk_1, \mathcal{S}_1, m_1) \\
&\approx \mathbf{Game}_\ell^{\mathsf{Real}}(spk_1, \mathcal{S}_1, m_1)
\end{aligned}$$

Therefore, the adversary's distinguishing advantage between the real games $\mathbf{Game}_0^{\mathsf{Real}}$ and $\mathbf{Game}_\ell^{\mathsf{Real}}$, which correspond to challenge bit $b = 0$ and $b = 1$ respectively, is bounded by the cumulative indistinguishability across the hybrid games, and is negligible under the assumed cryptographic assumptions. This completes the proof of *Theorem 1*.

Lemma 1: For each $\kappa \in [1, \xi]$, $\mathbf{Game}_\kappa$ is indistinguishable from $\mathbf{Game}_{\kappa-1}$ if RR-KDPKE scheme is IND-CCA secure. Specifically, we have $|\Pr[\mathcal{E}_\kappa] - \Pr[\mathcal{E}_{\kappa-1}]| \leq n \cdot \mathsf{Adv}_{\mathsf{PKE},\mathcal{B}}^{\mathsf{IND-CCA}}$.

Lemma 2: For each $\kappa \in [\xi + 1, \ell]$, $\mathbf{Game}_\kappa$ is indistinguishable from $\mathbf{Game}_{\kappa-1}$ if RR-KDPKE scheme is AI-CCA secure and WR-CCA secure. Specifically, for any FBME adversary $\mathcal{A}$ distinguishing between the two games, there exists either an AI-CCA adversary $\mathcal{B}$ or a WR-CCA adversary $\mathcal{B}'$ such that $|\Pr[\mathcal{E}_\kappa] - \Pr[\mathcal{E}_{\kappa-1}]| \leq n^2 \cdot (\mathsf{Adv}_{\mathsf{PKE},\mathcal{B}}^{\mathsf{AI-CCA}} + \mathsf{Adv}_{\mathsf{PKE},\mathcal{B}'}^{\mathsf{WR-CCA}})$.

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:12:53 UTC from IEEE Xplore. Restrictions apply.

We can similarly prove the *Lemma 1* & *Lemma 2* as the way in [28]. Due to space constraints, detailed proofs of these two lemmas are presented in the appendix. Combining the above arguments, we conclude $|\Pr[\mathcal{E}_0^{\mathsf{Real}}] - \Pr[\mathcal{E}_\ell^{\mathsf{Real}}]| \leq 2 \cdot \mathsf{Adv}_{\Sigma,\mathcal{A}}^{\mathsf{SUF}} + n^2 \cdot \ell \cdot (\mathsf{Adv}_{\mathsf{PKE},\mathcal{B}}^{\mathsf{AI-CCA}} + \mathsf{Adv}_{\mathsf{PKE},\mathcal{B}'}^{\mathsf{WR-CCA}}) + \mathsf{Adv}_{\hat{\mathsf{H}},\mathcal{A}}^{\mathsf{PRF}} + \mathsf{Adv}_{\Sigma,\mathcal{A}}^{\mathsf{UL}} \leq 2 \cdot \mathsf{Adv}_{\Sigma,\mathcal{A}}^{\mathsf{SUF}} + n^3 \cdot (\mathsf{Adv}_{\mathsf{PKE},\mathcal{B}}^{\mathsf{AI-CCA}} + \mathsf{Adv}_{\mathsf{PKE},\mathcal{B}'}^{\mathsf{WR-CCA}}) + \mathsf{Adv}_{\hat{\mathsf{H}},\mathcal{A}}^{\mathsf{PRF}} + \mathsf{Adv}_{\Sigma,\mathcal{A}}^{\mathsf{UL}}$. ■

Theorem 2: FBME achieves authenticity if the CBDH assumption holds under the random oracle model.

Proof: Suppose there exists an adversary $\mathcal{A}$ with a non-negligible advantage ϵ against the FBME scheme in the authenticity security model. We construct a simulator $\mathcal{B}$ that solves the CBDH problem with a non-negligible advantage. Specifically, $\mathcal{B}$ receives a random instance of CBDH problem $(g, g^x, g^y, g^z) \in \mathbb{G}^4$ to compute $e(g,g)^{xyz}$ interacting with $\mathcal{A}$.

Setup: $\mathcal{B}$ selects two collusion-resistant hash functions $\mathsf{H}_1 : \{0,1\}^* \to \mathbb{Z}_p^*$, $\mathsf{H}_2 : \mathbb{Z}_p^* \to \mathbb{G}$, and two collusion-resistant key derivation functions $\mathsf{H} : \mathbb{G} \to \{0,1\}^\eta$, $\hat{\mathsf{H}} : \mathbb{G}_T \to \{0,1\}^\eta$ for $\eta \in \mathsf{poly}(\lambda)$, which $\mathsf{H}_2, \hat{\mathsf{H}}$ are modeled as random oracles. For each sender, it picks $\beta, \theta_1, \theta_2 \in \mathbb{Z}_p^*$, computes $spk = (E_1 = g_1^{\theta_1} g_2^{\theta_2}, E_2 = g^{x\beta})$, and implicitly sets $ssk = (x\beta, \theta_1, \theta_2)$. Similarly, for each receiver, it generates key pairs (rpk_j, rsk_j) follows $\mathsf{RKGen}(pp)$, in which it picks $\gamma_j \in \mathbb{Z}_p^*$, computes $D_{3,j} = g^{y\gamma_j}$, and implicitly sets $\varphi_j = y\gamma_j$. It then follows $\mathsf{Setup}(\lambda, n)$ for the remaining parameters. Finally, $\mathcal{B}$ sends $pp = (\mathbb{G}, p, e, g_1, g_2, \mathsf{H}_1, \mathsf{H}_2, \mathsf{H}, \hat{\mathsf{H}}, \Sigma)$ to $\mathcal{A}$.

Query Phase: $\mathcal{A}$ queries the following oracle adaptively.

- $\mathcal{O}_{\mathsf{H}_2}$: $\mathcal{B}$ maintains a list L to respond to H_2 queries. If query $\alpha_l \in \mathbb{G}$ is in a tuple $\langle \alpha_l, h_l, \zeta_l, d_l \rangle \in L$, then return h_l. Otherwise, it chooses a new random exponent $\zeta_l \in \mathbb{Z}_p^*$, and tosses a biased coin $d_l \in \{0,1\}$ such that $\Pr[d_l = 0] = \delta$, which will be determined later. If $d_l = 0$, computes $h_l = g^{z \cdot \zeta_l}$; otherwise, computes $h_l = g^{\zeta_l}$. $\mathcal{B}$ adds the tuple $\langle \alpha_l, h_l, \zeta_l, d_l \rangle$ to the L, and responds to $\mathcal{A}$ by setting $\mathsf{H}_2(\alpha_l) = h_l$.
- $\mathcal{O}_{\hat{\mathsf{H}}}$: The list $\hat{L}$ is maintained by $\mathcal{B}$ to store the tuples of $(K_{\mathsf{s},j}, \hat{h}_j)$. If the query $K_{\mathsf{s},j}$ had been queried, then $\mathcal{B}$ returns the $\hat{h}_j$; otherwise, $\mathcal{B}$ randomly chooses $\hat{h}_j \in \{0,1\}^\eta$, adds the new tuple $(\alpha_l, \hat{h}_j)$ into $\hat{L}$ and responds to $\mathcal{A}$ with $\hat{\mathsf{H}}(K_{\mathsf{s},j}) = \hat{h}_j$.
- $\mathcal{O}_{\mathsf{Cor}_1}(spk)$: $\mathcal{B}$ updates $\mathsf{U}_{\mathsf{cor}} \leftarrow \mathsf{U}_{\mathsf{cor}} \cup \{spk\}$ and returns ssk such that $(spk, ssk) \in \mathsf{U}_{\mathsf{s}}$.
- $\mathcal{O}_{\mathsf{Cor}_2}(rpk_j)$: $\mathcal{B}$ updates $\mathsf{U}_{\mathsf{cor}} \leftarrow \mathsf{U}_{\mathsf{cor}} \cup \{rpk_j\}$ and returns rsk_j such that $(rpk_j, rsk_j) \in \mathsf{U}_{\mathsf{r}}$.
- $\mathcal{O}_{\mathsf{Enc}_1}(pp, spk, \mathcal{S}, m)$: To encrypt m for a public key set of receivers $\mathcal{S} = \{rpk_{\phi_1}, rpk_{\phi_2}, \ldots, rpk_{\phi_\ell}\}$, $\mathcal{B}$ generates a signature key pair $(ak, vk) \leftarrow \Sigma.\mathsf{KGen}(\lambda)$. Pick $w \in \mathbb{Z}_p^*$ and compute $(V_1, V_2) = (g_1^w, g_2^w)$. Then, it computes $\alpha_l = \mathsf{H}_1(V_1, V_2, vk)$, and looks up L to find the entry $\langle \alpha_l, h_l, \zeta_l, d_l \rangle$. If $d_l = 0$, it aborts; otherwise, for each $rpk_{\phi_i} \in \mathcal{S}$, computes $K_{\mathsf{s},\phi_i} = e(g^{x\beta}, D_{3,\phi_i})^{\zeta_l}$, looks up $\hat{L}$ to find the entry $(K_{\mathsf{s},\phi_i}, \hat{h}_{\phi_i})$. If there is no such entry, randomly chooses $\hat{h}_{\phi_i} \in \mathbb{Z}_p^*$, adds $(K_{\mathsf{s},\phi_i}, \hat{h}_{\phi_i})$ to $\hat{L}$ and sets $\hat{\mathsf{H}}(K_{\mathsf{s},\phi_i}) = \hat{h}_{\phi_i}$. It proceeds with the remaining computations as specified in the original encryption algorithm. Finally, it return ciphertext $ct = (vk, C = (V_1, V_2, \{A_{\tau(i)}\}_{i \in [1,\ell]}), \varsigma)$.

Forgery: $\mathcal{A}$ sends (spk^*, rpk^*, ct^*) to $\mathcal{B}$. Suppose that $\mathcal{A}$ wins the security game, then $\mathcal{B}$ performs the following steps as a response. It first computes $\alpha^* = \mathsf{H}_1(V_1, V_2, vk)$, look up $\hat{L}$ to find the entry $\langle \alpha^*, h^*, \zeta^*, d^* \rangle$. If $d^* = 1$, $\mathcal{B}$ aborts. If not, we know that $K_{\mathsf{s},\phi_i}^* = e(g^{x\beta}, D_{3,\phi_i}^*)^{z \cdot \zeta^*} = e(g^{x\beta}, g^{y\gamma_{\phi_i}})^{z \cdot \zeta^*} = (e(g,g)^{xyz})^{\beta\gamma_{\phi_i} \cdot \zeta^*}$. It takes a random tuple $(K_{\mathsf{s},\phi_i}, \hat{h}_{\phi_i})$ from $\hat{L}$ and computes $e(g,g)^{xyz} = \hat{h}_{\phi_i}^{1/(\beta\gamma_{\phi_i}\zeta^*)}$.

Analysis: Here, abt denotes the event where $\mathcal{B}$ aborts during the game, which occurs in two cases. Let q_C be the number of ciphertext queries and abt_1 denote the event that $d_l = 0$ in ciphertext queries. The probability that abt_1 does not occur is $\Pr[\neg\mathsf{abt}_1] = (1-\delta)^{q_C}$. Let abt_2 denote the event that $d_l = 1$ in the challenge, then $\Pr[\neg\mathsf{abt}_2] = \delta$. Then the probability that $\mathcal{B}$ does not abort is given by $\Pr[\neg\mathsf{abt}] = \Pr[\neg\mathsf{abt}_1] \cdot \Pr[\neg\mathsf{abt}_2] = (1-\delta)^{q_C}\delta$. Setting $\delta = \frac{1}{q_c+1}$ maximizes this probability, yielding $\Pr[\neg\mathsf{abt}] \approx \frac{1}{(q_C+1)e}$, which is non-negligible since q_C is polynomial in size and e is the natural logarithm base. Therefore, the probability that $\mathcal{B}$ solves the CBDH problem is $\Pr[b' = b] = \frac{1}{2} + \epsilon \cdot \Pr[\neg\mathsf{abt}]$, which is non-negligible if ϵ and $\Pr[\neg\mathsf{abt}]$ are non-negligible. So, we have that $\mathsf{Adv}_{\mathcal{A},\mathsf{FBME}}^{\mathsf{Auth}}(\lambda) \leq \frac{1}{2} \cdot \mathsf{Adv}_{\mathcal{B},\mathbb{G}}^{\mathsf{CBDH}}(\lambda)$. ■

Theorem 3: FBME achieves accountability if the NIZK proof system is knowledge extractable and the DL assumption holds.

Proof: The accountability intuitively follows from the underlying properties: demonstrating forgeries that deceive either the receiver (sender binding) or the moderator (receiver binding) implies the capability to create franking signatures without the sender secret key. We present the proof in the appendix. ■

Theorem 4: FBME achieves deniability if the underlying NIZK proof system has a zero-knowledge property and the DDH assumption holds.

Proof: The adversary without the sender secret key (s, θ_1, θ_2) generates an ill-formed encapsulated ciphertext c, samples $k_{\mathsf{J}} \in \mathbb{G}$ by running $\mathsf{Forge}(pp, pk_{\mathsf{J}}, spk, m)$. Finally, it calls $\mathsf{NIZK.Prove}$ to generate a NIZK proof. The goal is to demonstrate the indistinguishability of outputs from Frank and Forge. We present the proof in the appendix. ■

B. Security Analysis of MBAC

MBAC effectively mitigates various security threats by leveraging the security properties of FBME. First, MBAC ensures data confidentiality through FBME's privacy property, allowing only authorized receivers with valid secret keys to decrypt the ciphertext, thus preventing unauthorized access from both external and internal adversaries. Moreover, MBAC preserves receiver anonymity within FBME's privacy, ensuring that neither the service provider nor unauthorized users can deduce the set of privileged receivers from the ciphertext or reporting process, which is essential for privacy in multi-receiver settings. This anonymity also prevents the service provider from linking specific senders and receivers during individual communications, even with knowledge of the public keys of all registered users. In addition, MBAC resists impersonation attacks by leveraging FBME's authenticity, ensuring only legitimate data senders can produce valid ciphertexts, and preventing malicious parties from injecting forged data.

For content moderation, MBAC provides a verifiable reporting mechanism via FBME's accountability. Sender binding

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:12:53 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON OF COMPUTATIONAL COST

Schemes	Sender key generation	Receiver key generation	Judge key generation	Encryption	Decryption	Report	Judge
RBME [6]	$e_0 + e_h$	$2e_0 + e_h$	-	$2\ell e_p + (\ell^2 + 3)e_0$	$3e_p + \ell e_0 + 6e_h$	-	-
PSME [7]	$e_1 + e_h$	$2e_0 + e_h$	-	$2\ell e_p + (\ell + 1)e_0 + 6e_1 + (3\ell + 3)e_h$	$5e_p + 2e_0 + 6e_h$	-	-
Reclean [11]	e_0	$3e_0$	$3e_0$	$\ell e_p + (6\ell + 14)e_0 + (2\ell + 2)e_h$	$e_0 + e_h$	$(3\ell + 9)e_0 + e_h$	$(3\ell + 8)e_0 + e_h$
FBME	$3e_0$	$5e_0$	e_0	$\ell e_p + (3\ell + 3)e_0 + (\ell + 2)e_h$	$e_p + 3e_0 + 4e_h + 2\ell e_x$	$11e_0 + e_h$	$13e_0 + e_h$

TABLE III
COMPARISON OF STORAGE COST

Schemes	Sender key	Receiver key	Judge key	Ciphertext	Report
RBME [6]	$\|\mathbb{G}\|$	$2\|\mathbb{G}\|$	-	$(\ell + 3)\|\mathbb{G}\| + (\ell + 1)\|\mathbb{Z}_p^*\|$	-
PSME [7]	$\|\mathbb{H}\|$	$3\|\mathbb{G}\|$	-	$\|\mathbb{G}\| + 3\|\mathbb{H}\| + (2\ell + 1)\|\mathbb{Z}_p^*\| + \|m\|$	-
Reclean [11]	$\|\mathbb{G}\| + \|\mathbb{Z}_p^*\|$	$3\|\mathbb{G}\| + \|\mathbb{Z}_p^*\|$	$3\|\mathbb{G}\| + \|\mathbb{Z}_p^*\|$	$(5\ell + 15)\|\mathbb{G}\| + (\ell + 7)\|\mathbb{Z}_p^*\|$	$(\ell + 8)\|\mathbb{G}\| + (\ell + 3)\|\mathbb{Z}_p^*\|$
FBME	$2\|\mathbb{G}\| + 3\|\mathbb{Z}_p^*\|$	$3\|\mathbb{G}\| + 5\|\mathbb{Z}_p^*\|$	$\|\mathbb{G}\| + \|\mathbb{Z}_p^*\|$	$(\ell + 3)\|\mathbb{G}\| + \ell\|m\|$	$10\|\mathbb{G}\|$

prevents senders from altering ciphertexts to evade moderation, while receiver binding ensures receivers cannot submit fraudulent reports to frame innocent senders. Any report of malicious content is cryptographically tied to its sender, making the reporting process tamper-proof and resistant to abuse. Lastly, MBAC offers post-compromise protection through FBME's deniability. In case of a breach, senders can deny public attribution of specific data unless explicitly verified by the service provider, thus protecting their reputation and mitigating potential repercussions. By incorporating these robust security properties, MBAC effectively resists various attack vectors.

VII. PERFORMANCE EVALUATION

In this section, we evaluate the performance of FBME and compare it with representative state-of-the-art multi-receiver ME schemes, including RBME [6] and PSME [7], as well as the content-moderated data sharing scheme Reclean [11].

A. Theoretical Analysis

Let e_0 (or e_1), e_p, and e_h denote the execution time of one exponentiation in the group $\mathbb{G}$ (or $\mathbb{H}$), one pairing operation, and one hash operation in the group, respectively. Let e_x denote a symmetric operation (e.g., XOR), which is much faster than group operations. Let $|\mathbb{G}|$ (or $|\mathbb{H}|$) and $|\mathbb{G}_T|$ be the size of a single group element in $\mathbb{G}$ (or $\mathbb{H}$) and $\mathbb{G}_T$, respectively. Let ℓ be the size of the receiver set $\mathcal{S}$, and $|m|$ the message length. Theoretical comparisons of computation and storage costs are given in Tables II and III.

As shown in Table II, RBME and PSME require up to 2ℓ pairing operations during encryption, with RBME involving $\mathcal{O}(\ell^2)$ exponentiations due to its per-recipient ciphertext structure. In contrast, FBME and Reclean require only ℓ pairings, but Reclean incurs more exponentiations as it computes one for each receiver's public key, resulting in higher overall complexity. While the decryption algorithm involves scanning all ℓ ciphertext components to locate the correct one, each check requires only lightweight symmetric operations without repeated exponentiations or pairings, even the worst-case overhead is thus $2\ell e_x$. Since e_x is significantly cheaper than any group operation, this linear cost has negligible impact. For fairness, our implementation includes this worst-case traversal in the decryption time measurements depicted in Section VII-B, and FBME still outperforms Reclean in decryption efficiency. For reporting, FBME verifies one NIZK proof and performs one symmetric decryption, leading to constant cost per report. In contrast, Reclean's reporting cost scales with ℓ due to per-receiver group elements. As shown in Table III, FBME achieves linear ciphertext size and constant report size, whereas Reclean's ciphertext and report scale with ℓ, leading to higher bandwidth overhead.

B. Experimental Analysis

We utilize jPBC [37] to implement the experimental codes. Specifically, we use the supersingular (SS) curve with a 768-bit base field (SS768) for symmetric bilinear pairings, and MNT224 curves for asymmetric bilinear pairings [21], both providing approximately 96-bit security [38]. The algorithms are executed on a laptop equipped with 2.3 GHz Intel Core i5-6200 U CPU and 8 GB of memory. The code for our experiment is available at https://github.com/adslab-cn/FBME_Code. Fig. 8 illustrates the computational and storage overhead across the four schemes with varying numbers of receivers ranging from 10 to 50. Since RBME and PSME do not incorporate content moderation, our comparison for the JKGen and Judge algorithms is limited to Reclean and FBME. Moreover, as Reclean only supports the desired content moderation function with a single specified receiver, we extend it to accommodate multiple receivers for a fair comparison.

1) Key Generation: First, we present the experimental results and analysis for key generation from the perspectives of the data sender, data receiver, and moderator.

- *Sender Key Generation:* As shown in Fig. 8(a), FBME demonstrates a moderate computational overhead of around 82.6 ms. Reclean exhibits the lowest computational overhead at approximately 27.8 ms, while RBME and PSME incur higher overhead at 110.8 ms and 139.6 ms, respectively. Regarding storage overhead, as shown in Fig. 8(e), FBME requires 722 bytes due to its enhanced security mechanisms. Reclean has a lower storage overhead at 338 bytes, with RBME and PSME showing even lower values at 192 bytes and 168 bytes, respectively.
- *Receiver Key Generation:* As depicted in Fig. 8(a), FBME and RBME exhibit similar computational overheads, with

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:12:53 UTC from IEEE Xplore. Restrictions apply.

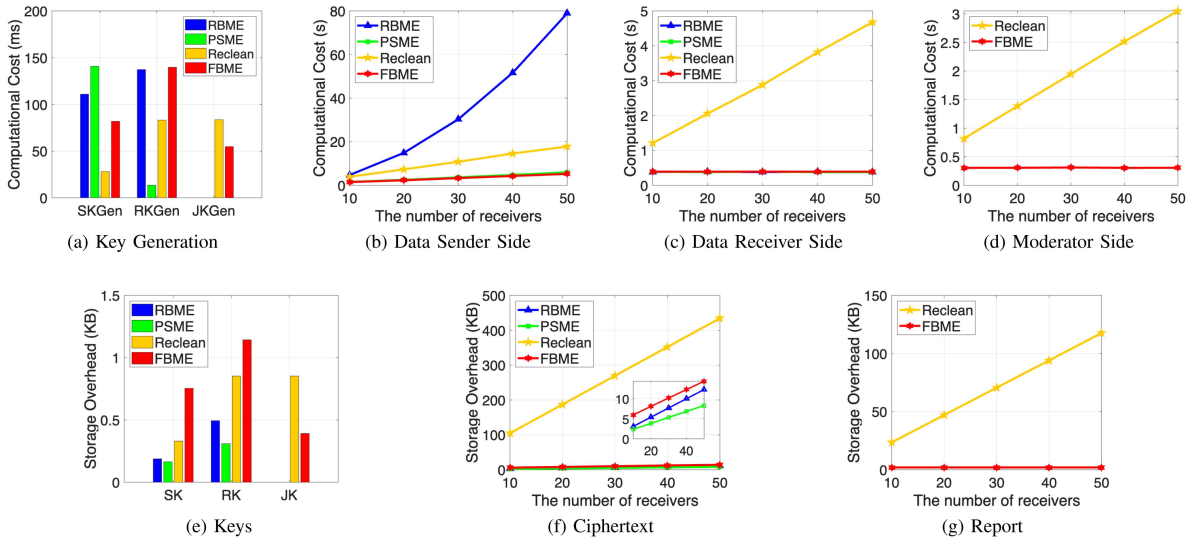


Fig. 8. Experimental results.

FBME at approximately 139.1 ms and RBME at around 138.1 ms. PSME demonstrates the lowest computational overhead at approximately 13.4 ms due to efficient asymmetric group computation, while Reclean shows a moderate overhead of around 82.4 ms. For storage, as shown in Fig. 8(e), FBME incurs the highest overhead at approximately 1.14 KB. In contrast, PSME exhibits the lowest storage overhead at 317 bytes, followed by RBME at 506 bytes and Reclean at 873 bytes.

- *Judge Key Generation:* As shown in Fig. 8(a), FBME has a lower computational overhead of around 54.8 ms, compared to Reclean's 83.2 ms. In terms of storage, as illustrated in Fig. 8(e), FBME incurs a lower overhead of approximately 401 bytes, significantly less than Reclean's 873 bytes, highlighting its storage efficiency.

2) Data Sender Side: The computational overhead on the data sender side increases with the number of receivers. As shown in Fig. 8(b), PSME has the lowest overhead, from 1.06 s to 4.15 s as the receiver count grows from 10 to 50. FBME exhibits slightly higher but comparable overhead, from 1.48 s to 5.25 s, indicating scalability with manageable costs. To adapt Reclean for multi-receiver settings, we employ the optimal extension technique suggested in AGMF [12], which generalizes the single-receiver relation from AMF [10] while preserving content moderation functionality. As a result, Reclean follows a linear trend in computational overhead, due to the NIZK proof generation proportional to the number of receivers, ranging from 3.96 s to 17.81 s. In contrast, RMBE shows a quadratic growth in overhead with the number of receivers, surging from 4.75 s to 81.15 s. For storage, ciphertext size also scales with the number of receivers. As shown in Fig. 8(f), PSME requires the least storage, increasing linearly from 2.39 KB to 8.32 KB as the receiver count rises from 10 to 50, and RBME follows a similar trend but with slightly higher costs, ranging from 3.18 KB to 12.32 KB. FBME exhibits moderate linear growth, from 5.97 KB to 14.38 KB. In contrast, Reclean shows a much steeper increase, from about 104.68 KB to 434.37 KB, making FBME significantly more efficient in storage, especially as the receiver set scales.

3) Data Receiver Side: The computational overhead on the data receiver side includes the costs of decrypting data and verifying the NIZK proof. As shown in Fig. 8(c), RBME and PSME exhibit comparable and stable costs of approximately 385.93 ms and 383.81 ms, respectively. In contrast, Reclean displays a linear increase in computational overhead due to complex proof verification, rising from about 1.21 s to 4.68 s as the number of receivers grows. FBME, however, achieves a lower stable computational overhead at around 392 ms, making it particularly suitable for resource-limited receivers.

4) Moderator Side: As shown in Fig. 8(d), FBME maintains stable computational overhead on the moderator side at around 306.2 ms, regardless of the number of receivers, demonstrating efficiency in the judging process. In contrast, Reclean shows a linear increase in overhead as the number of receivers grows, primarily driven by the NIZK proof verification, ranging from approximately 0.82 s to 3.05 s. The report's storage overhead is another key metric for evaluating the moderator's performance. As shown in Fig. 8(g), Reclean's overhead rises significantly from 6.15 KB to 23.26 KB as the NIZK proof size grows with the number of receivers. In contrast, FBME's overhead remains constant at about 2.02 KB, regardless of the number of receivers.

5) Summary: Experimental results demonstrate that FBME achieves encrypted content moderation without incurring heavy computational and storage overhead. Its performance on the data sender and receiver sides is comparable to previous schemes, while FBME significantly outperforms existing approaches in terms of report size and the efficiency of moderation verification on the moderator side.

VIII. Conclusion

To address the need for bilateral access control and the challenge of moderating harmful content in privacy-preserving cloud-based data sharing, we present MBAC, a privacy-aware multi-receiver bilateral access control framework with encrypted content moderation. MBAC introduces FBME, a novel cryptographic primitive that strengthens anonymous multi-receiver ME with CCA security, while supporting

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:12:53 UTC from IEEE Xplore. Restrictions apply.

verifiable reporting to mitigate harmful content. FBME ensures accountability by holding senders responsible for malicious content and provides deniability to protect sender privacy in post-compromise scenarios. Formal security analysis demonstrates FBME's resilience against various security threats, and experimental results highlight MBAC's practicality in achieving strong security and functionality with reasonable performance.

References

[1] R. Cramer and V. Shoup, "A practical public key cryptosystem provably secure against adaptive chosen ciphertext attack," in *Proc. 18th Annu. Int. Cryptology Conf. Adv. Cryptolo*, 1998, pp. 13–25.

[2] D. Boneh, C. Gentry, and B. Waters, "Collusion resistant broadcast encryption with short ciphertexts and private keys," in *Proc. 25th Annu. Int. Conf. Adv. Cryptology*, 2005, pp. 258–275.

[3] D. Boneh, G. Di Crescenzo, R. Ostrovsky, and G. Persiano, "Public key encryption with keyword search," in *Proc. Adv. Cryptol.-EUROCRYPT,*, 2004, pp. 506–522.

[4] Q. Huang and H. Li, "An efficient public-key searchable encryption scheme secure against inside keyword guessing attacks," *Inf. Sci.*, vol. 403, pp. 1–14, 2017.

[5] G. Ateniese, D. Francati, D. Nuñez, and D. Venturi, "Match me if you can: Matchmaking encryption and its applications," in *Proc. Adv. Cryptology–CRYPTO,*2019, pp. 701–731.

[6] A. Wu, D. Feng, M. Zhang, A. Yang, and J. Chi, "Privacy-preserving bilateral multi-receiver matching with revocability for mobile social networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 11080–11090, Dec. 2024.

[7] J. Sun, G. Xu, T. Zhang, X. Yang, M. Alazab, and R. H. Deng, "Privacy-aware and security-enhanced efficient matchmaking encryption," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 4345–4360, 2023.

[8] "Messenger secret conversations technical whitepaper," 2017. [Online]. Available: https://about.fb.com/wp-content/uploads/2016/07/messenger-secret-conversations-technical-whitepaper.pdf

[9] P. Grubbs, J. Lu, and T. Ristenpart, "Message franking via committing authenticated encryption," in *Proc. Adv. Cryptology–CRYPTO,* 2017, pp. 66–97.

[10] N. Tyagi, P. Grubbs, J. Len, I. Miers, and T. Ristenpart, "Asymmetric message franking: Content moderation for metadata-private end-to-end encryption," in *Proc. Adv. Cryptology–CRYPTO*, 2019, pp. 222–250.

[11] P. Jiang, B. Qiu, and L. Zhu, "Report when malicious: Deniable and accountable searchable message-moderation system," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 1597–1609, 2022.

[12] J. Lai, G. Zeng, Z. Huang, S. M. Yiu, X. Mu, and J. Weng, "Asymmetric group message franking: Definitions and constructions," in *Proc. Adv. Cryptology–EUROCRYPT,* 2023, pp. 67–97.

[13] J. Chen, Y. Li, J. Wen, and J. Weng, "Identity-based matchmaking encryption from standard assumptions," in *Proc. Adv. Cryptology–ASIACRYPT*, 2022, pp. 394–422.

[14] J. Chen, H. W. Lim, S. Ling, H. Wang, and H. Wee, "Shorter identity-based encryption via asymmetric pairings," *Designs, codes Cryptogr.*, vol. 73, pp. 911–947, 2014.

[15] Y. Zhang, J. Chen, J. Ning, Q. Chu, L. Wang, and G. Zhang, "Flexible privacy-preserving data computing with bilateral access control for cloud-assisted IoT," *IEEE Trans. Dependable Secure Comput.*, vol. 22, pp. 3166–3178, May/Jun. 2025.

[16] A. Wu, W. Luo, J. Weng, A. Yang, and J. Wen, "Fuzzy identity-based matchmaking encryption and its application," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 5592–5607, 2023.

[17] W. Huang, A. Wu, S. Xu, G. Xu, and W. Wu, "EASNs: Efficient anonymous social networks with enhanced security and high scalability," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 796–806, 2025.

[18] S. Xuet al., "Match in my way: Fine-grained bilateral access control for secure cloud-fog computing," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 1064–1077, Mar.-Apr. 2022.

[19] S. Xu, J. Ning, X. Huang, J. Zhou, and R. H. Deng, "Server-aided bilateral access control for secure data sharing with dynamic user groups," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4746–4761, 2021.

[20] J. Ma, S. Xu, J. Ning, X. Huang, and R. H. Deng, "Catch me if you can: A secure bilateral access control system with anonymous credentials," *IEEE Trans. Serv. Comput.*, vol. 16, no. 6, pp. 4444–4455, Nov.-Dec. 2023.

[21] S. Agrawal and M. Chase, "FAME: Fast attribute-based message encryption," in *Proc. 2017 ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 665–682.

[22] Y. Dodis, P. Grubbs, T. Ristenpart, and J. Woodage, "Fast message franking: From invisible salamanders to encryptment," in *Proc. Adv. Cryptology–CRYPTO*, 2018, pp. 155–186.

[23] R. Issa, N. Alhaddad, and M. Varia, "Hecate: Abuse reporting in secure messengers with sealed sender," in *Proc. 31st USENIX Secur. Symp. (USENIX Secur.* 22), 2022, pp. 2335–2352.

[24] R. Cramer and V. Shoup, "Universal hash proofs and a paradigm for adaptive chosen ciphertext secure public-key encryption," in *Proc. Lecture Notes in Comput. Sci.*, 2002, pp. 45–64.

[25] J. Cui, B. Li, H. Zhong, G. Min, Y. Xu, and L. Liu, "A practical and efficient bidirectional access control scheme for cloud-edge data sharing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 2, pp. 476–488, Feb. 2022.

[26] J. Rompel, "One-way functions are necessary and sufficient for secure signatures," in *Proc. 22nd Annu. ACM Symp. Theory Comput.*, 1990, pp. 387–394.

[27] K. Kurosawa and Y. Desmedt, "A new paradigm of hybrid encryption scheme," in *Proc. Adv. Cryptology–CRYPTO*, 2004, pp. 426–442.

[28] B. Libert, K. G. Paterson, and E. A. Quaglia, "Anonymous broadcast encryption: Adaptive security and efficient constructions in the standard model," in *Proc. Public Key Cryptography–PKC*, 2012, pp. 206–224.

[29] E. Ghadafi, N. P. Smart, and B. Warinschi, "Groth–Sahai proofs revisited," in *Proc. Public Key Cryptography–PKC*, 2010, pp. 177–192.

[30] H. Krawczyk, "Cryptographic extraction and key derivation: The HKDF scheme," in *Proc. Adv. Cryptology–CRYPTO 2010*, T. Rabin, Ed., 2010, pp. 631–648.

[31] D. Boneh, E. Shen, and B. Waters, "Strongly unforgeable signatures based on computational Diffie-Hellman," in *Proc. Lecture Notes in Comput. Sci.*, 2006, pp. 229–240.

[32] T. Okamoto, "An efficient divisible electronic cash scheme," in *Proc. Adv. Cryptology—CRYPT0'95, 1995*, pp. 438–451.

[33] D. Chaum and T. P. Pedersen, "Wallet databases with observers," in *Proc. Adv. Cryptology—CRYPTO'92*, 1992, pp. 89–105.

[34] C.-P. Schnorr, "Efficient identification and signatures for smart cards," in *Proc. Adv. Cryptology—CRYPTO'89*, 1990, pp. 239–252.

[35] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Proc. Conf. Theory Application Cryptographic Techn.*, 1986, pp. 186–194.

[36] "A graduate course in applied cryptography," 2023. [Online]. Available: https://toc.cryptobook.us/

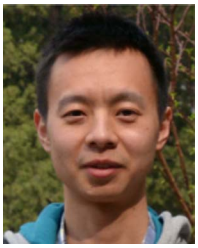
[37] A. D. Caro and V. Iovino, "jPBC: Java pairing based cryptography," in *Proc. IEEE Symp. Comput. Commun.*, 2011, pp. 850–855.

[38] Q. Zhang, S. Zhao, L. Xi, W. Feng, and D. Feng, "Mdaak: A flexible and efficient framework for direct anonymous attestation on mobile devices," in *Proc. 16th Int. Conf. Inf. Commun. Secur.*, 2015, pp. 31–48.

Chao Wang is currently working toward the PhD degree with the School of Cyberspace Security, Beijing University of Posts and Telecommunications. Her research interests include data security in cloud computing.

Willy Susilo (Fellow, IEEE) received the PhD degree in computer science from the University of Wollongong, Australia. He is currently an Australian Laureate fellow, distinguished professor and the Head of the School of Computing and Information Technology and the Director of the Institute of Cybersecurity and Cryptology, University of Wollongong. He was awarded the UOW Alumni Award for Research and Scholarship in 2024 and the prestigious Australian Research Council (ARC) Laureate Fellowship in 2023, the Vice-Chancellor's Research Excellence Award for Researcher of the Year in 2016 and the ARC Future Fellowship in 2009. He is the Editor-in-Chief of the Elsevier's Computer Standards and Interfaces and MDPI's Information journal. He was a program committee member in dozens of international conferences. He is currently an associate editor in several international journals, including the *IEEE Transactions on Dependable and Secure Computing* and *the International Journal of Information Security* (Springer). He has authored or coauthored more than 500 research papers in the area of cybersecurity and cryptology. His work has been cited more than 30,000 times in Google Scholar. His main research interests include cybersecurity, cryptography, and information security.

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:12:53 UTC from IEEE Xplore. Restrictions apply.



Yudi Zhang received the master's degree from the Hubei University of Technology, China, in 2017, and the Ph.D. degree from Wuhan University, China, in 2020. He is currently a lecturer with the School of Computing and Information Technology, University of Wollongong, Australia, where he previously was an associate research fellow from 2021 to 2023. His main research interests include cryptography and information security, in particular, cryptographic protocols.

Yumei Li received the PhD degree in statistics from Nanjing Normal University, China, in 2021. She previously held a lecturer position with the Hubei University of Technology in Wuhan, China. She is currently working toward the second PhD degree in cyber security with the University of Wollongong, Australia. Her primary research interests include public key cryptography and its applications in cloud computing and blockchain technologies.

Qinlong Huang (Member, IEEE) received the PhD degree in information security from the School of Computer Science, Beijing University of Posts and Telecommunications (BUPT), China, in 2014. From 2019 to 2020, he was a visiting scholar with Virginia Tech, USA. He is currently an associate professor with the School of Cyberspace Security, BUPT, China. He is the Principal investigator of several research projects funded by National Natural Science Foundation of China, China Computer Federation. He is also the TPC of several IEEE conferences, such as IEEE GLOBECOM, IEEE TrustCom. He has authored or coauthored more than 30 papers in *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Services Computing*, *IEEE Transactions on Cloud Computing*, *IEEE Transactions on Computational Biology and Bioinformatics*. His research interests include applied cryptography and data security.

Authorized licensed use limited to: Sri Venkateswara College of Engineering. Downloaded on January 27,2026 at 14:12:53 UTC from IEEE Xplore. Restrictions apply.